

LEETCODE SQL TOP 50

Complete Interview Preparation Handbook

SQL Theory · Optimised Solutions · Advanced Patterns · 35 Interview Q&A;

50

Problems
Covered

9

Topic
Chapters

35

Interview
Q & A

3

Bugs
Fixed

For Software Engineers · SQA Engineers · Data Analysts · Data Engineers · Backend Developers

Prepared: June 2026 · MySQL · All 50 Problems Covered

CHAPTER 0 ·

Executive Summary and How to Use This Handbook

This handbook is a complete SQL interview preparation resource built from an expert-level analysis of the LeetCode SQL Top 50 repository. It covers all 50 problems from first principles with optimised solutions, deep SQL theory, nine advanced SQL patterns, a 35-question interview Q&A; across three levels, a debugging checklist, three revision plans, a FAANG strategy section, and an optimisation guide.

Chapter	Content
I	SQL Fundamentals — SELECT, WHERE, NULL traps, ORDER BY, DISTINCT
II	Aggregation and Grouping — GROUP BY, HAVING, Conditional Aggregation
III	Joins — INNER, LEFT, CROSS, SELF, Anti-join, Date-Range JOIN
IV	Subqueries and Common Table Expressions
V	Window Functions — RANK, DENSE_RANK, LAG, LEAD, Rolling Windows
VI	Advanced SQL Patterns — Gap-and-Island, Session Detection, Median, MoM Growth, Pivot, Running Total, Top-N, Deduplication, Full Outer Join
VII	Problem Deep-Dive — Easy Problems (25 problems)
VIII	Problem Deep-Dive — Medium Problems (15 problems)
IX	Problem Deep-Dive — Hard Problems (10 problems)
X	Interview Preparation — Cheat Sheet, 35 Q&A;, Debugging, Revision Plans, FAANG Strategy
XI	Key Takeaways and Repository Audit Summary

How to Use This Handbook

- 1-Day Revision: Read Chapter X (cheat sheet + Q&A; + patterns), then skim all problem analyses.
- 3-Day Revision: Day 1 Theory Ch I-VI, Day 2 Easy and Medium problems, Day 3 Hard problems and Interview Prep.
- 7-Day Mastery: Follow the structured day-by-day plan inside Chapter X.
- Pre-Interview (2 hours): Jump to Chapter X Cheat Sheet and Top 10 Patterns table.
- Full Study: Read every chapter in order from I to XI for comprehensive preparation.

CHAPTER I ·

SQL Fundamentals

1.1 Logical Execution Order

The database engine does NOT run clauses in written order. The logical execution sequence is: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. This explains why a SELECT alias cannot be used in WHERE (WHERE runs before SELECT), and why aggregate functions are forbidden in WHERE (aggregation happens after WHERE).

● Execution Order Template

```
SELECT col1, COUNT(*) AS cnt
FROM   table_name
WHERE  condition           -- runs BEFORE GROUP BY
GROUP BY col1
HAVING COUNT(*) > 5        -- runs AFTER GROUP BY
ORDER BY cnt DESC
LIMIT 10;
```

Interview Insight — Execution Order

- Q: "Why can't you use a SELECT alias in WHERE?" A: WHERE runs before SELECT, so the alias doesn't exist yet.
- Q: "Can you use an alias in ORDER BY?" A: Yes — ORDER BY runs after SELECT.
- Q: "Can you filter on an aggregate in WHERE?" A: No — use HAVING.

1.2 WHERE — Row Filtering and the NULL Trap

WHERE filters individual rows before aggregation. The most dangerous interview trap in SQL is NULL handling: NULL compared with any value using = or != returns UNKNOWN, which WHERE treats as FALSE, silently dropping those rows.

● SQL

```
-- NULL TRAP — most common SQL interview mistake
SELECT name FROM customer WHERE referee_id != 2;
-- WRONG: NULL != 2 = UNKNOWN → NULL rows are silently excluded!

-- CORRECT:
SELECT name FROM customer
WHERE referee_id != 2 OR referee_id IS NULL;

-- NULL-safe alternative:
SELECT name FROM customer WHERE IFNULL(referee_id, 0) != 2;

-- BETWEEN is inclusive on both ends
SELECT * FROM orders
WHERE order_date BETWEEN '2023-01-01' AND '2023-12-31';

-- LIKE patterns
SELECT * FROM products WHERE name LIKE '%laptop%'; -- contains
SELECT * FROM users WHERE email LIKE '%.com'; -- ends with
```

Warning — NULL Is Not a Value

- NULL = NULL → UNKNOWN (not TRUE). Use IS NULL.
- NULL != 2 → UNKNOWN (not FALSE). Add OR col IS NULL when NULLs should be kept.
- NULL in arithmetic → NULL. Use IFNULL(col, 0) to provide defaults.
- Affected LeetCode problems: #584, #577, #1978, #619, #176.

1.3 ORDER BY, LIMIT and DISTINCT

● SQL

```
-- Sort multi-column: primary salary DESC, tiebreak name ASC
SELECT * FROM employees ORDER BY salary DESC, name ASC;

-- Pagination with OFFSET
SELECT * FROM products ORDER BY created_at DESC LIMIT 10 OFFSET 20;

-- Second highest salary — LIMIT OFFSET approach
SELECT DISTINCT salary FROM employee
ORDER BY salary DESC LIMIT 1 OFFSET 1;

-- DISTINCT — deduplicate result rows
SELECT DISTINCT author_id FROM views WHERE author_id = viewer_id;
```

CHAPTER II ·

Aggregation and Grouping

2.1 Aggregate Functions

Function	NULL Handling	Typical Use Case
COUNT(*)	Counts ALL rows including NULLs	Total row count
COUNT(col)	Skips NULL values	Count non-empty entries
COUNT(DISTINCT col)	Unique non-NULL values only	Distinct entity count
SUM(col)	Ignores NULLs	Revenue, quantities
AVG(col)	Ignores NULLs; denominator = non-NULLs	Average salary, score
MIN(col) / MAX(col)	Ignores NULLs	Date range, price extremes

2.2 GROUP BY and HAVING

GROUP BY partitions rows into groups. Aggregate functions operate per group. HAVING filters groups after aggregation — it is the WHERE clause for groups.

● SQL

```
-- Classes with 5+ students (LC #596)
SELECT class FROM courses
GROUP BY class
HAVING COUNT(student) >= 5;

-- Combined WHERE + HAVING
SELECT department, AVG(salary) AS avg_sal
FROM employees
WHERE hire_date > '2020-01-01' -- filter ROWS before grouping
GROUP BY department
HAVING AVG(salary) > 60000;      -- filter GROUPS after aggregating
```

WHERE vs HAVING — The Critical Difference

- WHERE: filters rows BEFORE GROUP BY. Cannot contain aggregate functions.
- HAVING: filters groups AFTER GROUP BY. Must use aggregate functions here.
- Common mistake: WHERE COUNT(*) > 5 → SQL syntax error. Correct: HAVING COUNT(*) > 5.

2.3 Conditional Aggregation — IF / CASE Inside Aggregates

Conditional aggregation counts or sums only rows meeting a condition, all in a single query pass. This is the "pivot" pattern.

● SQL

```
-- Monthly Transactions (LC #1193) – multiple metrics in one query
```

```
SELECT
    DATE_FORMAT(trans_date, '%Y-%m')      AS month,
    country,
    COUNT(id)                             AS trans_count,
    SUM(IF(state = 'approved', 1, 0))      AS approved_count,
    SUM(amount)                           AS trans_total_amount,
    SUM(IF(state = 'approved', amount, 0)) AS approved_total_amount
FROM   transactions
GROUP BY month, country;
```

```
-- Confirmation Rate (LC #1934) – proportion using AVG of 0/1
```

```
SELECT user_id,
       ROUND(AVG(IF(action = 'confirmed', 1, 0)), 2) AS confirmation_rate
FROM   signups s
LEFT JOIN confirmations c USING(user_id)
GROUP BY user_id;
```

CHAPTER III ·

Joins — Combining Data Across Tables

Join Type	Returns	When to Use	LC Examples
INNER JOIN	Rows matching in BOTH tables	Only care about matched data	#1068, #1661
LEFT JOIN	ALL left rows + matched right (NULL if no match)	Keep all from left table	#577, #1378, #1581
CROSS JOIN	All combinations — Cartesian product	Generate every pair combination	#1280
SELF JOIN	Table joined to itself using aliases	Hierarchy, consecutive comparison	#570, #197
FULL OUTER	All rows from both (MySQL: emulate with UNION)	Rarely needed; simulate in MySQL	Advanced

3.1 LEFT JOIN and the WHERE Conversion Danger

● SQL

```
-- Employee Bonus (LC #577): include employees with NO bonus
SELECT e.name, b.bonus
FROM   employee e
LEFT   JOIN bonus b ON e.empid = b.empid
WHERE  b.bonus < 1000 OR b.bonus IS NULL;

-- DANGER: WHERE b.bonus < 1000 alone DROPS the NULL rows!
-- NULL < 1000 = UNKNOWN → those rows are excluded.
-- Fix 1: add OR b.bonus IS NULL
-- Fix 2: move condition to ON clause:
LEFT JOIN bonus b ON e.empid = b.empid AND b.bonus < 1000
```

LEFT JOIN + WHERE Danger Zone

- After LEFT JOIN, unmatched right-table rows appear as NULLs.
- Filtering WHERE right.col < value drops those NULLs — silently converting to INNER JOIN!
- Always add OR right.col IS NULL when unmatched rows should be kept.
- Or move the filter to the ON clause — it won't drop unmatched rows there.

3.2 Anti-Join — Find Rows NOT in Another Table

● SQL

```
-- Visits with no transaction (LC #1581)
SELECT v.customer_id, COUNT(v.visit_id) AS count_no_trans
FROM   visits v
LEFT   JOIN transactions t ON t.visit_id = v.visit_id
WHERE  t.transaction_id IS NULL    -- keep ONLY unmatched rows
GROUP BY v.customer_id;

-- Alternative: NOT EXISTS (often clearer in interviews)
SELECT customer_id, COUNT(*) AS count_no_trans
FROM   visits v
WHERE  NOT EXISTS (
    SELECT 1 FROM transactions t WHERE t.visit_id = v.visit_id
)
GROUP BY customer_id;
```

3.3 Self Join and Cross Join

● SQL

```
-- Rising Temperature: today warmer than yesterday (LC #197)
SELECT w1.id
FROM   weather w1
JOIN   weather w2 ON DATEDIFF(w1.recorddate, w2.recorddate) = 1
WHERE  w1.temperature > w2.temperature;
-- Use DATEDIFF, NOT w2.id = w1.id - 1 (breaks if IDs have gaps!)

-- Students x Subjects: every combination with attendance count (LC #1280)
SELECT  s.student_id, s.student_name, sub.subject_name,
        COUNT(e.subject_name) AS attended_exams
FROM    students s
CROSS   JOIN subjects sub
LEFT    JOIN examinations e
        ON s.student_id = e.student_id
        AND e.subject_name = sub.subject_name
GROUP   BY s.student_id, s.student_name, sub.subject_name
ORDER   BY s.student_id, sub.subject_name;
```


CHAPTER IV ·

Subqueries and CTEs

4.1 Subquery Types

Type	Location	Example
Scalar	SELECT / WHERE / HAVING	(SELECT MAX(salary) FROM emp)
Multi-row IN	WHERE col IN (...)	WHERE dept IN (SELECT dept FROM t)
Row constructor	WHERE (a,b) IN (...)	WHERE (product_id, year) IN (SELECT ...)
Correlated	WHERE / HAVING	WHERE EXISTS (SELECT 1 FROM t WHERE t.id = outer.id)
Derived table	FROM clause	FROM (SELECT ...) alias WHERE alias.rnk <= 3

● SQL

```
-- Scalar subquery: Second Highest Salary (LC #176)
SELECT MAX(salary) AS SecondHighestSalary FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);

-- Row constructor: first-year product sales (LC #1070)
SELECT product_id, year AS first_year, quantity, price
FROM sales
WHERE (product_id, year) IN (
    SELECT product_id, MIN(year) FROM sales GROUP BY product_id
);

-- Relational division: bought ALL products (LC #1045)
SELECT customer_id FROM customer
GROUP BY customer_id
HAVING COUNT(DISTINCT product_key) = (SELECT COUNT(*) FROM product);
```

4.2 CTEs — WITH Clause

CTEs (Common Table Expressions) define named temporary result sets at the top of a query. They improve readability over nested subqueries and allow referencing the same result multiple times.

● SQL

```
-- CTE syntax
WITH cte1 AS (
    SELECT ...
    FROM ...
),
cte2 AS (
    SELECT ... FROM cte1 WHERE ...
)
SELECT * FROM cte2;

-- Department Top 3 Salaries using CTE (LC #185)
WITH ranked AS (
    SELECT d.name AS Department,
           e.name AS Employee,
           e.salary,
           DENSE_RANK() OVER (PARTITION BY d.id
                               ORDER BY e.salary DESC) AS rnk
    FROM   employee e
    INNER JOIN department d ON e.departmentid = d.id
)
SELECT Department, Employee, salary FROM ranked WHERE rnk <= 3;
```

CTE vs Subquery vs Temp Table

- CTE: Readable, no disk I/O, reusable within one query. Best for multi-step logic.
- Subquery: Good for simple one-off nesting. Hard to read when deeply nested.
- Temp Table: Actual disk storage. Use only for very large intermediate results.

CHAPTER V ·

Window Functions — Advanced Analytics

Window functions compute values across a set of rows related to the current row, **WITHOUT** collapsing them like **GROUP BY** does. They are the most powerful and most frequently tested SQL feature in senior interviews.

● Window Function Syntax

```
function_name() OVER (
    [PARTITION BY col1, col2]           -- reset per group
    [ORDER BY col3 ASC|DESC]           -- order within window
    [ROWS BETWEEN n PRECEDING
      AND CURRENT ROW]                -- frame (optional)
)
```

5.1 Ranking Functions

Function	Ties Handled?	Gaps After Tie?	Scores 100,100,90
ROW_NUMBER()	Assigns unique rank always	N/A	1, 2, 3
RANK()	Same rank for ties	YES	1, 1, 3
DENSE_RANK()	Same rank for ties	NO	1, 1, 2
NTILE(n)	Divides into n equal buckets	N/A	Percentile grouping

● SQL

```
-- Top 3 salaries per department (LC #185)
-- DENSE_RANK is correct: ties get same rank, no gap after
SELECT Department, Employee, salary
FROM (
    SELECT d.name AS Department,
           e.name AS Employee,
           e.salary,
           DENSE_RANK() OVER (PARTITION BY d.id
                               ORDER BY e.salary DESC) AS rnk
    FROM   employee e
    INNER JOIN department d ON e.departmentid = d.id
) t
WHERE rnk <= 3;
-- RANK()      → 1,1,3 for ties: rank-3 slot could be missed
-- DENSE_RANK() → 1,1,2 for ties: always 3 distinct salary levels
```

5.2 LAG and LEAD — Accessing Adjacent Rows

● SQL

```
-- LAG(col, n, default): value n rows BEFORE current
-- LEAD(col, n, default): value n rows AFTER current

-- Consecutive numbers using LAG – handles ID gaps (LC #180)
SELECT DISTINCT num AS ConsecutiveNums
FROM (
    SELECT num,
           LAG(num, 1) OVER (ORDER BY id) AS prev1,
           LAG(num, 2) OVER (ORDER BY id) AS prev2
    FROM   logs
) t
WHERE num = prev1 AND num = prev2;

-- Game Play Analysis: day after first login (LC #550)
SELECT ROUND(SUM(logged_next_day) / COUNT(DISTINCT player_id), 2) AS fraction
FROM (
    SELECT player_id,
           (DATEDIFF(event_date,
                     MIN(event_date) OVER (PARTITION BY player_id)) = 1
            ) AS logged_next_day
    FROM   activity
) t;
```

5.3 Rolling Windows and Running Totals

● SQL

```
-- 7-day rolling average — Restaurant Growth (LC #1321)
SELECT DISTINCT visited_on,
       SUM(daily) OVER (
         ORDER BY visited_on
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
       ) AS amount,
       ROUND(AVG(daily) OVER (
         ORDER BY visited_on
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
       ), 2) AS average_amount
FROM (
  SELECT visited_on, SUM(amount) AS daily
  FROM   customer GROUP BY visited_on
) daily_totals
ORDER BY visited_on
LIMIT 18446744073709551615 OFFSET 6;
-- OFFSET 6 skips first 6 days (incomplete 7-day window)

-- Running total — Last Person to Fit in Bus (LC #1204)
SELECT person_name FROM (
  SELECT person_name,
         SUM(weight) OVER (
           ORDER BY turn
           ROWS UNBOUNDED PRECEDING
         ) AS cum_weight
  FROM   queue
) t
WHERE  cum_weight <= 1000
ORDER BY cum_weight DESC
LIMIT 1;
```

Window Functions vs Self-Joins — Always Prefer Window

- Self-join running total: $O(n^2)$ — for each row, scans all previous rows.
- `SUM() OVER (ROWS UNBOUNDED PRECEDING)`: $O(n \log n)$ — single ordered pass.
- `LAG/LEAD` replace consecutive-row self-joins and handle ID gaps correctly.
- In interviews: propose window function if you see a self-join for ordered comparison.

CHAPTER VI ·

Advanced SQL Patterns

This chapter covers patterns that go beyond individual LeetCode problems — the recurring templates that appear in data engineering, analytics, and senior engineering interviews. Mastering these patterns lets you solve entire classes of problems, not just individual questions.

Pattern 1: Gap and Island

Concept

The gap-and-island problem identifies consecutive groups (islands) of rows that share a property, separated by gaps where that property is absent. Classic examples: find periods of continuous user activity, find date ranges where a product was in stock, detect uninterrupted login streaks.

Core Technique

The standard approach uses `ROW_NUMBER()` to create a "group key": if rows are consecutive, then $(\text{date} - \text{ROW_NUMBER}())$ is constant within each island. Any gap breaks that constant, starting a new group.

● Gap and Island — Login Streaks

```
-- Example: find continuous login streaks per user
-- Table: logins(user_id, login_date)

WITH numbered AS (
    SELECT user_id,
           login_date,
           ROW_NUMBER() OVER (
               PARTITION BY user_id
               ORDER BY login_date
           ) AS rn
    FROM   logins
),
grouped AS (
    SELECT user_id,
           login_date,
           -- If dates are consecutive, (date - rn days) is constant
           DATE_SUB(login_date, INTERVAL rn DAY) AS grp_key
    FROM   numbered
)
SELECT user_id,
       MIN(login_date) AS streak_start,
       MAX(login_date) AS streak_end,
       COUNT(*)        AS streak_length
FROM   grouped
GROUP BY user_id, grp_key
ORDER BY user_id, streak_start;
```

Step-by-Step Breakdown

Step 1: Number rows per user ordered by date with ROW_NUMBER().

Step 2: Subtract that row number (as days) from the login date. If Jan 1, Jan 2, Jan 3 → rn 1,2,3 → grp_key
Dec 31, Dec 31, Dec 31 (same key = same island).

Step 3: A gap (e.g. Jan 5 missing) shifts the grp_key for Jan 6 forward, creating a new distinct key.

Step 4: GROUP BY (user_id, grp_key) then MIN/MAX date gives each island's range.

● Longest Streak per User

```
-- Variation: find the LONGEST streak per user
WITH numbered AS (
  SELECT user_id, login_date,
         ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date) AS rn
  FROM   logins
),
grouped AS (
  SELECT user_id, DATE_SUB(login_date, INTERVAL rn DAY) AS grp_key,
         COUNT(*) AS streak_len
  FROM   numbered
  GROUP BY user_id, grp_key
)
SELECT user_id, MAX(streak_len) AS longest_streak
FROM   grouped
GROUP BY user_id;
```

Interview Insight — Gap and Island

- Always clarify: are we looking for gaps (missing rows) or islands (consecutive groups)?
- The DATE_SUB(date, INTERVAL rn DAY) technique only works when the "step" is exactly 1 day.
- For arbitrary consecutive integers (not dates): use col - ROW_NUMBER() as the group key.
- LeetCode connection: LC #180 (Consecutive Numbers) is the simplest gap-and-island variant.
- FAANG variant: "Find users active for 3+ consecutive days in the past month."

Pattern 2: Session Detection

Concept

A session is a group of events by the same entity within a time window. If two consecutive events by the same user are more than N minutes apart, a new session begins. This pattern is fundamental in web analytics, clickstream analysis, and user behaviour modelling.

● Session Detection with 30-Minute Timeout

```

-- Table: events(user_id, event_time)
-- Sessions: gap of > 30 minutes = new session

WITH lagged AS (
    SELECT user_id,
           event_time,
           LAG(event_time) OVER (
               PARTITION BY user_id
               ORDER BY event_time
           ) AS prev_time
    FROM   events
),
session_flags AS (
    SELECT user_id, event_time,
           -- New session starts when gap > 30 min OR it's the first event
           CASE
               WHEN prev_time IS NULL
                OR TIMESTAMPDIFF(MINUTE, prev_time, event_time) > 30
               THEN 1 ELSE 0
           END AS is_new_session
    FROM   lagged
),
with_session_id AS (
    SELECT user_id, event_time,
           SUM(is_new_session) OVER (
               PARTITION BY user_id
               ORDER BY event_time
               ROWS UNBOUNDED PRECEDING
           ) AS session_id
    FROM   session_flags
)
SELECT user_id,
       session_id,
       MIN(event_time) AS session_start,
       MAX(event_time) AS session_end,
       COUNT(*)        AS events_in_session
FROM   with_session_id
GROUP BY user_id, session_id
ORDER BY user_id, session_id;

```

How Session IDs Work

- LAG() gets the previous event time per user.
- If gap > 30 min (or first event), flag it as a new session start (1), else 0.
- SUM(flag) OVER (ROWS UNBOUNDED PRECEDING) = running count of session starts = unique session ID.
- This is the standard technique used by Google Analytics, Mixpanel, and similar tools.
- TIMESTAMPDIFF(MINUTE, a, b) gives the difference in minutes between two datetimes.

Pattern 3: Median Calculation

Concept

MySQL has no built-in `MEDIAN()` function. The median is the middle value when sorted: the value at position $(N+1)/2$ for odd N , or the average of positions $N/2$ and $N/2+1$ for even N . Two common approaches: `PERCENTILE_CONT` (not in MySQL 8.0), or using `ROW_NUMBER` with `COUNT`.

● Median Salary — Three Approaches

```
-- Method 1: ROW_NUMBER approach (works in MySQL)
-- Find the middle row(s) by ranking from both ends
SELECT AVG(salary) AS median_salary
FROM (
    SELECT salary,
           ROW_NUMBER() OVER (ORDER BY salary ASC) AS rn_asc,
           ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn_desc,
           COUNT(*) OVER () AS total
    FROM employee
) t
WHERE rn_asc BETWEEN total/2 AND total/2 + 1
     AND rn_desc BETWEEN total/2 AND total/2 + 1;

-- Method 2: Percentile using NTILE
SELECT AVG(salary) AS median_salary
FROM (
    SELECT salary,
           NTILE(2) OVER (ORDER BY salary) AS tile
    FROM employee
) t
WHERE tile = 1
     OR salary = (
        SELECT MIN(salary) FROM (
            SELECT salary, NTILE(2) OVER (ORDER BY salary) AS tile
            FROM employee
        ) t2 WHERE tile = 2
    );

-- Method 3: Simplest for interviews
SELECT AVG(salary) AS median_salary
FROM (
    SELECT salary,
           ROW_NUMBER() OVER (ORDER BY salary) AS rn,
           COUNT(*) OVER () AS n
    FROM employee
) t
WHERE rn IN (FLOOR((n+1)/2), CEIL((n+1)/2));
```

Interview Insight — Median

- Never say "MySQL has a `MEDIAN()` function" — it does not.
- Method 3 (`FLOOR`/`CEIL`) is the cleanest to explain and works for odd and even N .
- For odd N : $\text{FLOOR}((5+1)/2)=3$ and $\text{CEIL}((5+1)/2)=3 \rightarrow$ same row $\rightarrow$ single median.
- For even N : $\text{FLOOR}((6+1)/2)=3$ and $\text{CEIL}((6+1)/2)=4 \rightarrow$ two middle rows $\rightarrow$ AVG.

- Follow-up: "Median salary per department?" → add PARTITION BY dept to both ROW_NUMBER calls.

Pattern 4: Month-over-Month Growth

Concept

Month-over-month (MoM) growth compares a metric in the current period to the same metric in the previous period. LAG() makes this clean: access the prior month's value on the current row and compute the percentage change.

● Month-over-Month Revenue Growth

```
-- Table: orders(order_id, order_date, amount)
-- Goal: monthly revenue with MoM growth %

WITH monthly AS (
  SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
         SUM(amount) AS revenue
  FROM   orders
  GROUP BY month
),
with_lag AS (
  SELECT month,
         revenue,
         LAG(revenue) OVER (ORDER BY month) AS prev_revenue
  FROM   monthly
)
SELECT month,
       revenue,
       prev_revenue,
       ROUND(
         (revenue - prev_revenue) * 100.0 / prev_revenue,
         2) AS mom_growth_pct
FROM   with_lag
ORDER BY month;

-- Handle NULL for first month (no previous period):
SELECT month, revenue,
       IFNULL(ROUND((revenue - prev_revenue)*100.0/prev_revenue, 2), 0)
       AS mom_growth_pct
FROM   with_lag
ORDER BY month;
```

Interview Insight — MoM Growth

- LAG(revenue) OVER (ORDER BY month) gets previous month revenue on the current row.
- Growth % = (current - previous) / previous × 100.
- First month has no LAG value → NULL. Use IFNULL to display 0 or N/A.
- Year-over-year: LAG(revenue, 12) OVER (ORDER BY month) — 12 months back.

- Common follow-up: "Which month had the highest growth?" → wrap in subquery, ORDER BY growth DESC LIMIT 1.

Pattern 5: Pivoting — Rows to Columns

Concept

Pivoting transforms distinct row values into column headers. MySQL has no native PIVOT keyword. The standard approach uses conditional aggregation: SUM(IF(category = 'X', value, 0)) AS X for each desired column.

● Pivot — Quarterly Revenue per Product

```
-- Table: sales(product, quarter, revenue)
-- Goal: one row per product, one column per quarter

SELECT
    product,
    SUM(IF(quarter = 'Q1', revenue, 0)) AS Q1,
    SUM(IF(quarter = 'Q2', revenue, 0)) AS Q2,
    SUM(IF(quarter = 'Q3', revenue, 0)) AS Q3,
    SUM(IF(quarter = 'Q4', revenue, 0)) AS Q4
FROM   sales
GROUP BY product
ORDER BY product;

-- Alternative with CASE WHEN (standard SQL):
SELECT
    product,
    SUM(CASE WHEN quarter = 'Q1' THEN revenue ELSE 0 END) AS Q1,
    SUM(CASE WHEN quarter = 'Q2' THEN revenue ELSE 0 END) AS Q2,
    SUM(CASE WHEN quarter = 'Q3' THEN revenue ELSE 0 END) AS Q3,
    SUM(CASE WHEN quarter = 'Q4' THEN revenue ELSE 0 END) AS Q4
FROM   sales
GROUP BY product;
```

Pivot Interview Notes

- MySQL has no PIVOT keyword — always use conditional aggregation.
- If the number of distinct column values is unknown at query time, you need dynamic SQL (PREPARE/EXECUTE) — mention this in interviews.
- LeetCode connection: LC #1193 (Monthly Transactions) is a pivot-style problem.
- Unpivot (columns to rows): use UNION ALL with a hardcoded value per column.

Pattern 6: Running Totals and Cumulative Metrics

Concept

A running total accumulates a metric row-by-row in a defined order. Window functions make this a single-pass $O(n \log n)$ operation. Common use cases: cumulative revenue, balance tracking, stock levels, user growth curves.

● Running Totals — Three Variants

```
-- Running revenue total by date
SELECT order_date,
       daily_revenue,
       SUM(daily_revenue) OVER (
         ORDER BY order_date
         ROWS UNBOUNDED PRECEDING
       ) AS cumulative_revenue
FROM (
  SELECT order_date, SUM(amount) AS daily_revenue
  FROM   orders
  GROUP BY order_date
) daily
ORDER BY order_date;

-- Running total RESET per month (PARTITION BY)
SELECT order_date,
       DATE_FORMAT(order_date, '%Y-%m') AS month,
       amount,
       SUM(amount) OVER (
         PARTITION BY DATE_FORMAT(order_date, '%Y-%m')
         ORDER BY order_date
         ROWS UNBOUNDED PRECEDING
       ) AS monthly_running_total
FROM   orders
ORDER BY order_date;

-- Running COUNT of distinct users
SELECT event_date,
       SUM(COUNT(DISTINCT user_id)) OVER (
         ORDER BY event_date
         ROWS UNBOUNDED PRECEDING
       ) AS cumulative_users
FROM   activity
GROUP BY event_date;
```

Frame Clause Reference

- ROWS UNBOUNDED PRECEDING: from first row to current row — classic running total.
- ROWS BETWEEN 6 PRECEDING AND CURRENT ROW: 7-row rolling window (LC #1321).
- ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING: 3-row centred moving average.
- RANGE vs ROWS: RANGE groups rows with equal ORDER BY values; ROWS is physical row count.
- In interviews, ROWS is almost always what you want — RANGE can produce unexpected results with ties.

Pattern 7: Top-N per Group — Deep Dive

Concept

Top-N per group is the single most tested Hard SQL pattern in interviews. It means: for each group (department, category, user), return the N rows with the highest (or lowest) value of some metric.

● Top-N Template + When to Use Each Function

```
-- Template: Top N rows per group
SELECT *
FROM (
    SELECT *,
           DENSE_RANK() OVER (
               PARTITION BY group_col
               ORDER BY metric_col DESC
           ) AS rnk
    FROM   source_table
) t
WHERE rnk <= N;

-- When to use which ranking function:

-- ROW_NUMBER: exactly N rows per group (no ties at boundary)
--   Use when: you want exactly 1 winner even if tied

-- RANK: N distinct rank positions, but may return < N rows
--   if ties skip a rank (1,1,3 → only 2 rows for top 2)

-- DENSE_RANK: N distinct value levels, may return > N rows
--   if tied at boundary. Standard choice for "top N salary levels"

-- Example: top 1 product per category by units sold
SELECT category, product, total_units
FROM (
    SELECT category, product,
           SUM(units) AS total_units,
           ROW_NUMBER() OVER (
               PARTITION BY category
               ORDER BY SUM(units) DESC
           ) AS rn
    FROM   sales
    GROUP BY category, product
) t
WHERE rn = 1;
```

ROW_NUMBER vs RANK vs DENSE_RANK — Decision Guide

- Need exactly 1 winner per group? → ROW_NUMBER(). Tiebreak by a secondary column in ORDER BY.
- Top N salary levels (ties included at boundary)? → DENSE_RANK().
- Top N positions (skip after ties)? → RANK().
- In LeetCode #185 (Department Top 3 Salaries): DENSE_RANK is correct because "top 3" means top 3 distinct salary values, not top 3 individual employees.

- Always state your choice and reasoning in interviews.

Pattern 8: Deduplication

Concept

Deduplication removes duplicate rows while keeping one representative row per group. The "keep" criterion matters: smallest ID, most recent date, highest value. Three main approaches: DELETE self-join, ROW_NUMBER filter, or NOT IN with MIN subquery.

● Deduplication — Three Methods

```
-- Method 1: DELETE self-join — keep smallest ID (LC #196)
DELETE p1 FROM person p1
INNER JOIN person p2
    ON p1.email = p2.email
    AND p1.id > p2.id;

-- Method 2: ROW_NUMBER — keep most recent (highest id)
DELETE FROM person
WHERE id NOT IN (
    SELECT id FROM (
        SELECT id,
            ROW_NUMBER() OVER (
                PARTITION BY email
                ORDER BY id DESC
            ) AS rn
        FROM person
    ) t WHERE rn = 1
);

-- Method 3: SELECT deduplicated rows (read-only)
SELECT *
FROM (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY email
            ORDER BY id ASC
        ) AS rn
    FROM person
) t
WHERE rn = 1;
```

Deduplication Interview Notes

- Always clarify: which row to keep? smallest id, largest id, most recent date?
- Method 1 (DELETE self-join) is the most efficient for actual deletion.
- Method 3 (ROW_NUMBER SELECT) is useful when you want to read deduplicated data without modifying the table.
- MySQL prevents directly subquerying the same table in a DELETE — always wrap in a derived table.

Pattern 9: Full Outer Join in MySQL

Concept

MySQL does not support FULL OUTER JOIN directly. To get all rows from both tables (matched and unmatched), use UNION of a LEFT JOIN and a RIGHT JOIN.

● Full Outer Join Simulation

```
-- Full outer join: all rows from both tables
SELECT a.id AS a_id, b.id AS b_id, a.val, b.val
FROM   table_a a
LEFT   JOIN table_b b ON a.id = b.id
UNION
SELECT a.id, b.id, a.val, b.val
FROM   table_a a
RIGHT  JOIN table_b b ON a.id = b.id;

-- Rows only in A (not in B):
SELECT * FROM table_a a
LEFT   JOIN table_b b ON a.id = b.id
WHERE  b.id IS NULL;

-- Rows only in B (not in A):
SELECT * FROM table_a a
RIGHT  JOIN table_b b ON a.id = b.id
WHERE  a.id IS NULL;

-- Rows in either but NOT BOTH (symmetric difference):
SELECT a.id, b.id FROM table_a a
LEFT   JOIN table_b b ON a.id = b.id WHERE b.id IS NULL
UNION
SELECT a.id, b.id FROM table_a a
RIGHT  JOIN table_b b ON a.id = b.id WHERE a.id IS NULL;
```

Interview Note

- Interviewers sometimes ask "how do you do a FULL OUTER JOIN in MySQL?" — know this.
- UNION (not UNION ALL) to avoid duplicating matched rows that appear in both halves.
- The symmetric difference query is useful for finding mismatches between two datasets.

CHAPTER VI ·

Problem Deep Dive — Easy Problems

#1757

Recyclable and Low Fat Products

SELECT · WHERE

Easy

Concept	Basic AND filter on two columns
Assessment	OPTIMAL

● SQL

```
SELECT product_id
FROM   products
WHERE  low_fats = 'Y' AND recyclable = 'Y';
```

Analysis

- Correct and optimal — single table scan $O(n)$.
- Composite index on (low_fats, recyclable) enables $O(\log n + k)$ on large tables.
- No improvements needed.

#595

Big Countries

WHERE · OR

Easy

Concept	Compound condition with OR
Assessment	OPTIMAL

● SQL

```
SELECT name, population, area
FROM   world
WHERE  area >= 3000000 OR population >= 25000000;
```

Analysis

- Correct. OR joins two independent criteria for "big country."
- Alternative — UNION: useful when separate indexes exist on area and population.
- OR approach is simpler and preferred when no separate indexes are present.

#1148

Article Views I

DISTINCT · WHERE · ORDER BY

Easy

Concept	Self-comparison within one row
---------	--------------------------------

Assessment	OPTIMAL
------------	---------

● SQL

```
SELECT DISTINCT author_id AS id
FROM   views
WHERE  author_id = viewer_id
ORDER BY author_id;
```

Analysis

- DISTINCT is essential — an author may view their own article multiple times.
- ORDER BY author_id is required by the problem specification.
- Clean and optimal.

#1683

Invalid Tweets

WHERE · LENGTH

Easy

Concept	String length check
---------	---------------------

Assessment	CORRECT — minor improvement available
------------	---------------------------------------

● SQL

```
SELECT tweet_id
FROM   tweets
WHERE  LENGTH(content) > 15;
```

Analysis + Improvement

- Correct. LENGTH() counts bytes — fine for ASCII-only content.
- For international text or emojis use CHAR_LENGTH(), which counts characters.
- CHAR_LENGTH is the safer production and interview choice.

#584

Find Customer Referee

NULL Handling

Easy

Key Trap	NULL != 2 = UNKNOWN, not FALSE — NULL rows are silently dropped
----------	---

Assessment	CORRECT
------------	---------

● SQL

```
-- Your solution (CORRECT):
SELECT name FROM customer
WHERE referee_id != 2 OR referee_id IS NULL;

-- WRONG (common mistake):
SELECT name FROM customer WHERE referee_id != 2;
-- NULL != 2 = UNKNOWN → WHERE excludes UNKNOWN → NULLs disappear!

-- NULL-safe alternative:
SELECT name FROM customer WHERE IFNULL(referee_id, 0) != 2;
```

Top Interview NULL Trap

- This is the #1 SQL interview NULL trap.
- `NULL != value` always evaluates to UNKNOWN, not FALSE.
- Always explicitly handle NULLs: `col != value OR col IS NULL`.
- `IFNULL(col, default)` replaces NULL before comparison — cleaner but less explicit.

#620

Not Boring Movies

WHERE · MOD · ORDER BY

Easy

Concept	Odd-ID filter with modulo operator
Assessment	OPTIMAL

● SQL

```
SELECT *
FROM   cinema
WHERE  id % 2 != 0
      AND description != 'boring'
ORDER BY rating DESC;
```

Analysis

- `id % 2 != 0` selects odd IDs. Equivalent: `MOD(id, 2) = 1`.
- `ORDER BY rating DESC` as specified.
- Optimal — single-pass filter with sort.

#1378

Replace Employee ID With Unique Identifier

LEFT JOIN

Easy

Pattern	Include all from A, look up optional data from B
Assessment	OPTIMAL

● SQL

```

SELECT eu.unique_id, e.name
FROM   employees e
LEFT   JOIN employeeuni eu ON e.id = eu.id;

```

LEFT JOIN Pattern

- LEFT JOIN keeps ALL employees. Employees with no unique_id get NULL — correct.
- INNER JOIN would be WRONG — it drops employees with no unique ID mapping.
- "Include all from A, look up optional from B" → always LEFT JOIN.

#1068

Product Sales Analysis I

INNER JOIN · Multi-table

Easy

Note	Your solution uses implicit join syntax — explicit preferred in interviews
Assessment	CORRECT

SQL

```

-- Your solution (implicit join – valid but not preferred):
SELECT p.product_name, s.year, s.price
FROM   sales s
JOIN   product p ON p.product_id = s.product_id;

-- Preferred explicit style:
SELECT p.product_name, s.year, s.price
FROM   sales s
INNER  JOIN product p ON s.product_id = p.product_id;

```

Implicit vs Explicit JOIN

- Implicit: FROM t1, t2 WHERE t1.id = t2.fk — still seen in legacy code.
- Explicit: INNER JOIN ... ON ... — clearer intent, standard practice.
- Both produce identical results; explicit syntax is the professional standard in interviews.

#1581

Customer Who Visited but Did Not Make Any Transactions

Anti-Join

Easy

Pattern	LEFT JOIN + WHERE right.id IS NULL = anti-join
Assessment	OPTIMAL

SQL

```
SELECT v.customer_id, COUNT(v.visit_id) AS count_no_trans
FROM   visits v
LEFT   JOIN transactions t ON t.visit_id = v.visit_id
WHERE  t.transaction_id IS NULL  -- only unmatched rows
GROUP BY v.customer_id;
```

Anti-Join Pattern

- LEFT JOIN + WHERE right.id IS NULL = the canonical anti-join.
- This keeps ONLY visits that have no matching transaction.
- COUNT(v.visit_id) counts no-transaction visits per customer.
- Alternative: NOT EXISTS subquery — equally valid, sometimes clearer.

#197

Rising Temperature

Self Join · DATEDIFF

Easy

Note	Your id-based join assumes sequential IDs — DATEDIFF is more robust
Assessment	CORRECT — LAG version is better

SQL

```
-- Your solution (self-join with DATEDIFF — correct):
SELECT w1.id
FROM   weather w1
JOIN   weather w2 ON DATEDIFF(w1.recorddate, w2.recorddate) = 1
WHERE  w1.temperature > w2.temperature;

-- Optimal — LAG avoids O(n^2) self-join:
SELECT id FROM (
    SELECT id, temperature, recorddate,
           LAG(temperature) OVER (ORDER BY recorddate) AS prev_temp,
           LAG(recorddate)  OVER (ORDER BY recorddate) AS prev_date
    FROM   weather
) t
WHERE  temperature > prev_temp
AND    DATEDIFF(recorddate, prev_date) = 1;
```

DATEDIFF vs id+1

- id+1 FAILS if IDs have gaps — DATEDIFF checks actual calendar days.
- LAG() operates on row positions by ORDER BY, not ID values — robust to any ID scheme.
- LAG approach is O(n log n); self-join is O(n^2) — prefer LAG in interviews.

#1661

Average Time of Process per Machine

Self Join · AVG

Easy

Pattern	Join start+end pairs then compute duration
Assessment	OPTIMAL

● SQL

```
SELECT a1.machine_id,
       ROUND(AVG(a2.timestamp - a1.timestamp), 3) AS processing_time
FROM   activity a1
JOIN   activity a2
      ON a1.machine_id = a2.machine_id
      AND a1.process_id = a2.process_id
      AND a1.activity_type = 'start'
      AND a2.activity_type = 'end'
GROUP BY a1.machine_id;
```

Pair-Matching Self Join

- Join start and end rows on (machine_id, process_id) pair.
- AVG(end.ts - start.ts) averages duration across all processes per machine.
- Alternative: SUM(IF(type='end', ts, -ts)) / COUNT(DISTINCT process_id).

#577

Employee Bonus

LEFT JOIN · NULL Keep

Easy

Pattern	Keep all left rows + filter with NULL awareness
Assessment	OPTIMAL

● SQL

```
SELECT e.name, b.bonus
FROM   employee e
LEFT   JOIN bonus b ON e.empid = b.empid
WHERE  b.bonus < 1000 OR b.bonus IS NULL;
```

Classic LEFT JOIN + NULL Pattern

- LEFT JOIN: all employees. No bonus row → b.bonus = NULL.
- b.bonus < 1000: keeps low-bonus employees.
- b.bonus IS NULL: keeps no-bonus employees.
- Definitive pattern for: "condition OR no matching row."

#1075

Project Employees I

LEFT JOIN · AVG · ROUND

Easy

Concept	Average experience per project
Assessment	OPTIMAL

● SQL

```
SELECT  p.project_id,
        ROUND(AVG(e.experience_years), 2) AS average_years
FROM    project p
LEFT    JOIN employee e ON e.employee_id = p.employee_id
GROUP   BY p.project_id;
```

Analysis

- LEFT JOIN ensures projects with no employees still appear.
- ROUND(AVG(...), 2) formats to 2 decimal places.
- INNER JOIN would silently drop empty projects — usually wrong.

#2356

Number of Unique Subjects Taught by Each Teacher

COUNT DISTINCT

Easy

Concept	Count unique values per group
Assessment	OPTIMAL

● SQL

```
SELECT  teacher_id,
        COUNT(DISTINCT subject_id) AS cnt
FROM    teacher
GROUP   BY teacher_id;
```

Analysis

- Perfect one-liner. COUNT(DISTINCT) is the exact right tool.
- Follow-up: "Teachers with only 1 subject" → HAVING COUNT(DISTINCT subject_id) = 1.

#596

Classes More Than 5 Students

GROUP BY · HAVING

Easy

Concept	Filter groups by row count
Assessment	OPTIMAL

● SQL

```
SELECT class
FROM   courses
GROUP BY class
HAVING COUNT(student) >= 5;
```

Analysis

- HAVING COUNT >= 5 filters at the group level — cannot use WHERE here.
- COUNT(student) skips NULLs. COUNT(*) counts all rows including NULLs.
- Follow-up: use COUNT(DISTINCT student) if a student can enroll multiple times.

#1729

Find Followers Count

GROUP BY · COUNT · ORDER BY

Easy

Concept	Count per group with ordering
Assessment	OPTIMAL

● SQL

```
SELECT  user_id,
        COUNT(follower_id) AS followers_count
FROM    followers
GROUP BY user_id
ORDER BY user_id;
```

Analysis

- Straightforward GROUP BY + COUNT. No improvements needed.
- Follow-up: "Users with above-average followers" → HAVING COUNT(*) > (SELECT AVG(fc) FROM (SELECT COUNT(*) AS fc FROM followers GROUP BY user_id) t).

#619

Biggest Single Number

MAX · HAVING · NULL-safe

Easy

Pattern	MAX of subquery converts empty result to NULL
Assessment	OPTIMAL

● SQL

```
SELECT MAX(num) AS num
FROM (
    SELECT num FROM mynumbers
    GROUP BY num
    HAVING COUNT(num) = 1
) AS singles;
-- MAX of empty set = NULL — correct when no single number exists.
```

NULL-safe Result Pattern

- Without the outer MAX: returns zero rows (not a NULL row) when no singles exist.
- MAX of an empty set returns NULL — exactly what the problem requires.
- General pattern: wrap subquery in MAX/MIN to produce NULL instead of empty set.

#1633

Percentage of Users Attended a Contest

Scalar Subquery · ROUND

Easy

Concept	Percentage using scalar subquery as denominator
Assessment	OPTIMAL

● SQL

```
SELECT  contest_id,
        ROUND(
            COUNT(DISTINCT user_id) * 100.0
            / (SELECT COUNT(*) FROM users),
            2) AS percentage
FROM    register
GROUP BY contest_id
ORDER BY percentage DESC, contest_id ASC;
```

Analysis

- 100.0 (not 100) forces floating-point division, not integer division.
- Scalar subquery runs once and is reused for all groups (optimizer caches it).
- ORDER BY percentage DESC, contest_id — secondary sort for tiebreaking.

#1211

Queries Quality and Percentage

Conditional AVG

Easy

Concept	AVG of ratio + AVG of 0/1 indicator
Assessment	OPTIMAL

● SQL

```
SELECT  query_name,
        ROUND(AVG(rating / position), 2)          AS quality,
        ROUND(AVG(IF(rating < 3, 1, 0)) * 100, 2) AS poor_query_percentage
FROM    queries
WHERE   query_name IS NOT NULL
GROUP BY query_name;
```

Analysis

- quality = AVG(rating/position) — average of the per-row ratio.

- $\text{poor_query_percentage} = \text{AVG}(\text{IF}(\text{rating} < 3, 1, 0)) \times 100 = \text{proportion} \times 100.$
- WHERE query_name IS NOT NULL prevents NULL from becoming a group key.

#1141

User Activity for Past 30 Days I

DATE · BETWEEN · COUNT DISTINCT

Easy

Concept	30-day date window with BETWEEN
Assessment	OPTIMAL

● SQL

```
SELECT activity_date AS day,
       COUNT(DISTINCT user_id) AS active_users
FROM   activity
WHERE  activity_date BETWEEN
       DATE_SUB('2019-07-27', INTERVAL 29 DAY)
       AND '2019-07-27'
GROUP BY activity_date;
```

Date Range Logic

- $\text{DATE_SUB}('2019-07-27', \text{INTERVAL } 29 \text{ DAY}) = 2019-06-28.$
- BETWEEN includes both endpoints → 30 days total (29-day interval + 1).
- If INTERVAL were 30 days, you would get 31 days — common off-by-one trap.
- Days with zero active users will not appear (no GROUP BY rows for empty days).

#1667

Fix Names in a Table

String Functions · CONCAT

Easy

Concept	Capitalize first letter, lowercase rest
Assessment	OPTIMAL

● SQL

```
SELECT user_id,
       CONCAT(
           UPPER(LEFT(name, 1)),
           LOWER(SUBSTRING(name, 2))
       ) AS name
FROM   users
ORDER BY user_id;
```

String Manipulation Pattern

- LEFT(name, 1) extracts first character. UPPER() capitalises it.
- SUBSTRING(name, 2) extracts from character 2 onward. LOWER() lowercases it.

- CONCAT joins them. MySQL alternative: `CONCAT(UCASE(LEFT(n,1)), LCASE(MID(n,2)))`.

#1527

Patients With a Condition

LIKE · REGEXP

Easy

Pattern	Match code at start of string OR after a space
Assessment	CORRECT

● SQL

```
-- Your LIKE solution (correct):
SELECT * FROM patients
WHERE conditions LIKE 'DIAB1%'      -- DIAB1 at start
   OR conditions LIKE '% DIAB1%';  -- DIAB1 after a space

-- Cleaner with REGEXP:
SELECT * FROM patients
WHERE conditions REGEXP '^(^| )DIAB1';
-- ^ = start of string, | = OR, space+DIAB1 = preceded by space
```

LIKE vs REGEXP

- LIKE with two conditions handles start and middle positions correctly.
- REGEXP `(^| )DIAB1` is more concise and maintainable.
- REGEXP is slower than LIKE on large tables — mention the tradeoff in interviews.

#196

Delete Duplicate Emails

DELETE · Self Join

Easy

Pattern	Delete the higher-ID duplicate, keep the lowest
Assessment	OPTIMAL

● SQL

```
DELETE p1
FROM   person p1
INNER JOIN person p2
      ON p1.email = p2.email
      AND p1.id > p2.id;
-- Keeps the row with SMALLEST id per email.
-- Deletes every row where a smaller id exists with the same email.
```

DELETE with Self-Join

- `p1.id > p2.id` ensures only the higher-ID duplicate is deleted.
- The lower-ID row never satisfies `p1.id > p2.id`, so it is kept.

- Clarify in interviews: "keep smallest id" vs "keep most recent" — requirements differ!

#1251

Average Selling Price

Date-Range JOIN · IFNULL

Easy

Pattern JOIN includes date range to select correct price tier

Assessment OPTIMAL

SQL

```
SELECT p.product_id,
       IFNULL(
         ROUND(SUM(p.price * u.units) / SUM(u.units), 2),
         0) AS average_price
FROM   prices p
LEFT   JOIN unitssold u
       ON p.product_id = u.product_id
       AND u.purchase_date BETWEEN p.start_date AND p.end_date
GROUP BY p.product_id;
```

Date-Constrained JOIN Pattern

- JOIN condition includes date range: purchase_date BETWEEN start AND end.
- This ensures the correct price tier is matched to each purchase.
- LEFT JOIN keeps products with no sales. IFNULL converts NULL result to 0.

#1327

List the Products Ordered in a Period

JOIN · DATE Filter · HAVING

Easy

Concept Filter by month and aggregate with threshold

Assessment OPTIMAL

SQL

```
SELECT p.product_name, SUM(o.unit) AS unit
FROM   products p
JOIN   orders o ON p.product_id = o.product_id
WHERE  DATE_FORMAT(o.order_date, '%Y-%m') = '2020-02'
GROUP BY p.product_id, p.product_name
HAVING SUM(o.unit) >= 100;
```

Analysis

- DATE_FORMAT(date, '%Y-%m') = '2020-02' filters to February 2020.
- Alternative: EXTRACT(YEAR_MONTH FROM date) = 202002 — MySQL-specific shortcut.
- HAVING SUM >= 100 filters to products meeting the minimum unit threshold.

CHAPTER VII ·

Problem Deep Dive — Medium Problems

#176

Second Highest Salary

Subquery · MAX · DENSE_RANK

Medium

Bug	Returns empty set instead of NULL when only 1 distinct salary exists
Fix	Wrap in IFNULL or use DENSE_RANK approach

● SQL

```
-- Your solution (correct but missing NULL edge case):
SELECT MAX(salary) AS SecondHighestSalary
FROM   employee
WHERE  salary < (SELECT MAX(salary) FROM employee);

-- IMPROVED – guaranteed NULL when no second salary:
SELECT IFNULL(
    (SELECT DISTINCT salary
     FROM   employee
     ORDER BY salary DESC
     LIMIT  1 OFFSET 1),
    NULL
) AS SecondHighestSalary;

-- BEST – generalises to Nth salary (change OFFSET to N-1):
SELECT MAX(salary) AS SecondHighestSalary FROM (
    SELECT salary,
           DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM   employee
) t WHERE rnk = 2;
```

Three Approaches — Know All Three

- Nested MAX: simple but returns empty result (not NULL) on edge case.
- LIMIT OFFSET: SELECT salary ... LIMIT 1 OFFSET 1. Wrap in IFNULL for NULL safety.
- DENSE_RANK: most flexible — change WHERE rnk = 2 to any N for Nth highest.
- Interview move: present DENSE_RANK and note it generalises to Nth salary easily.

#1280

Students and Examinations

CROSS JOIN · LEFT JOIN · GROUP BY

Medium

Pattern	Generate all combinations then count matches
Assessment	OPTIMAL

● SQL

```
SELECT  s.student_id, s.student_name,
        sub.subject_name,
        COUNT(e.subject_name) AS attended_exams
FROM    students s
CROSS   JOIN subjects sub
LEFT    JOIN examinations e
        ON s.student_id = e.student_id
        AND e.subject_name = sub.subject_name
GROUP   BY s.student_id, s.student_name, sub.subject_name
ORDER   BY s.student_id, sub.subject_name;
```

CROSS JOIN — All-Combinations Pattern

- CROSS JOIN: every student × every subject (Cartesian product).
- LEFT JOIN: matches actual attendance; NULL if not attended.
- COUNT(e.subject_name): 0 for unattended (NULLs not counted).
- This is the canonical "all combinations with zero-safe count" pattern.

#570

Managers with At Least 5 Direct Reports

Self Join · HAVING

Medium

Concept	Count reports per manager
Assessment	CORRECT — alternative clearer

● SQL

```
-- Your solution (self-join):
SELECT e1.name FROM employee e1
JOIN   employee e2 ON e1.id = e2.managerid
GROUP BY e2.managerid HAVING COUNT(*) >= 5;

-- Cleaner — IN subquery (preferred in interviews):
SELECT name FROM employee
WHERE id IN (
    SELECT managerid FROM employee
    GROUP BY managerid
    HAVING COUNT(*) >= 5
);
```

Self-Join vs Subquery

- Both are correct. IN+subquery reads as: "managers whose ID appears 5+ times."
- Self-join approach joins employees to their reports — equally valid.
- Prefer IN+subquery in interviews for clearer intent.

#1934

Confirmation Rate

Conditional AVG · LEFT JOIN

Medium

Pattern	AVG of 0/1 indicator = proportion
Assessment	OPTIMAL

● SQL

```
SELECT  s.user_id,
        ROUND(
            AVG(IF(c.action = 'confirmed', 1, 0)),
            2) AS confirmation_rate
FROM    signups s
LEFT    JOIN confirmations c ON s.user_id = c.user_id
GROUP BY s.user_id;
```

AVG of 0/1 Indicator Pattern

- AVG(IF(condition, 1, 0)) computes the proportion of true rows per group.
- LEFT JOIN: users with no confirmations have all NULLs.
- IF(NULL = 'confirmed', 1, 0) = 0, so their AVG = 0 — exactly correct.
- Cleaner than SUM/COUNT because NULL rows auto-contribute 0.

#1193

Monthly Transactions I

DATE_FORMAT · Conditional SUM · GROUP BY

Medium

Pattern	Multiple conditional metrics in one query pass
Assessment	OPTIMAL

● SQL

```
SELECT
    DATE_FORMAT(trans_date, '%Y-%m') AS month,
    country,
    COUNT(id) AS trans_count,
    SUM(IF(state = 'approved', 1, 0)) AS approved_count,
    SUM(amount) AS trans_total_amount,
    SUM(IF(state = 'approved', amount, 0)) AS approved_total_amount
FROM    transactions
GROUP BY month, country;
```

Multi-Metric Conditional Aggregation

- DATE_FORMAT for year-month grouping.
- SUM(IF(condition, 1, 0)): count only matching rows.
- SUM(IF(condition, amount, 0)): sum only matching amounts.
- GROUP BY both month AND country for per-country monthly breakdown.
- This pivot-style pattern is extremely common in data analyst interviews.

#1174

Immediate Food Delivery II

MIN per customer · Row Constructor

Medium

BUG	MIN(order_date) and MIN(pref_date) are computed independently — may be from different rows
Fix	Use row constructor subquery to find first-order row, then check its pref_date

SQL

```
-- YOUR SOLUTION HAS A BUG:
-- MIN(order_date) and MIN(customer_pref_delivery_date) are independent.
-- They may come from DIFFERENT orders for the same customer!

-- CORRECT approach:
SELECT ROUND(
    AVG(IF(order_date = customer_pref_delivery_date, 1, 0)) * 100,
    2) AS immediate_percentage
FROM delivery
WHERE (customer_id, order_date) IN (
    SELECT customer_id, MIN(order_date)
    FROM delivery
    GROUP BY customer_id
);
```

Bug Explanation

- Your code: MIN(order_date) and MIN(pref_date) computed as separate aggregates.
- Problem: MIN(pref_date) may be from a DIFFERENT order than MIN(order_date).
- Example: order Jan1 pref Jan1 (immediate) and order Jan5 pref Jan3 (non-immediate).
- MIN(pref_date) = Jan1 matches MIN(order_date) = Jan1 — false "immediate" result!
- Fix: identify the first order per customer, THEN check that specific row.

#550

Game Play Analysis IV

Window Function · DATEDIFF · Fraction

Medium

Pattern	MIN() OVER (PARTITION BY) for first-event access on every row
Assessment	OPTIMAL

● SQL

```
SELECT ROUND(
  SUM(logged_next_day) / COUNT(DISTINCT player_id),
  2) AS fraction
FROM (
  SELECT player_id,
    (DATEDIFF(
      event_date,
      MIN(event_date) OVER (PARTITION BY player_id)
    ) = 1) AS logged_next_day
  FROM activity
) t;
```

Window Function Mastery

- MIN(event_date) OVER (PARTITION BY player_id): each row sees its player's first login.
- DATEDIFF = 1: this row is exactly one day after first login → boolean 1 or 0.
- SUM(logged_next_day) counts players who logged in on day 2.
- O(n log n) window vs O(n^2) self-join — always choose window for performance.

#1164

Product Price at a Given Date

UNION · MAX date filter

Medium

Pattern	Two-part UNION: products with history + products without
Assessment	OPTIMAL

● SQL


```

SELECT product_id, new_price AS price
FROM   products
WHERE  (product_id, change_date) IN (
    SELECT product_id, MAX(change_date)
    FROM   products
    WHERE  change_date <= '2019-08-16'
    GROUP BY product_id
)
UNION
SELECT product_id, 10 AS price
FROM   products
WHERE  product_id NOT IN (
    SELECT product_id FROM products
    WHERE change_date <= '2019-08-16'
);

```

Two-Part UNION Pattern

- Part 1: Products with price history on/before the date → use latest price.
- Part 2: Products with NO history before the date → use default price of 10.
- Row constructor (product_id, change_date) IN (...) matches column pairs.
- UNION deduplicates — though no overlap should exist between the two parts.

#1341

Movie Rating

UNION ALL · Multi-sort subqueries

Medium

Key	Use UNION ALL not UNION — same value could appear in both parts
Assessment	CORRECT

SQL

```

(SELECT u.name AS results
FROM   users u
LEFT   JOIN movierating mr ON u.user_id = mr.user_id
GROUP BY u.user_id
ORDER BY COUNT(mr.user_id) DESC, u.name ASC
LIMIT 1)
UNION ALL
(SELECT m.title AS results
FROM   movies m
LEFT   JOIN movierating mr ON m.movie_id = mr.movie_id
WHERE  DATE_FORMAT(mr.created_at, '%Y-%m') = '2020-02'
GROUP BY m.movie_id
ORDER BY AVG(mr.rating) DESC, m.title ASC
LIMIT 1);

```

UNION ALL for Multi-Part Problems

- UNION ALL (not UNION): if the same name appears in both parts, UNION would deduplicate it.

- Part 1: user with most reviews — tiebreak: name ASC.
- Part 2: highest avg-rating movie in Feb 2020 — tiebreak: title ASC.
- Each subquery has its own ORDER BY + LIMIT inside parentheses — valid MySQL syntax.

#1321

Restaurant Growth

Rolling Window · Correlated Subquery

Medium**Bug**Your correlated subquery is $O(n^2)$ — $O(n \log n)$ window function is optimal**Fix**

Use SUM/AVG OVER (ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)

● SQL

```
-- Your solution: correct but  $O(n^2)$  correlated subquery

-- OPTIMAL with window function  $O(n \log n)$ :
SELECT DISTINCT visited_on,
       SUM(daily) OVER (
         ORDER BY visited_on
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
       ) AS amount,
       ROUND(AVG(daily) OVER (
         ORDER BY visited_on
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
       ), 2) AS average_amount
FROM (
  SELECT visited_on, SUM(amount) AS daily
  FROM   customer
  GROUP BY visited_on
) daily_totals
ORDER BY visited_on
LIMIT 18446744073709551615 OFFSET 6;
```

Correlated Subquery → Window Function Optimisation

- Correlated subquery: runs for EACH outer row → $O(n^2)$.
- Window ROWS BETWEEN 6 PRECEDING AND CURRENT ROW: $O(n \log n)$ — single pass.
- First aggregate by date (SUM per day), then apply rolling window.
- OFFSET 6 skips first 6 days that have incomplete 7-day windows.
- This optimisation question is a core data engineering interview topic.

#1204

Last Person to Fit in the Bus

Running Sum · Self Join

Medium

Pattern	Running total using SUM OVER (ROWS UNBOUNDED PRECEDING)
Assessment	CORRECT — window version is faster

● SQL

```
-- Your self-join solution (correct, O(n^2)):
```

```
SELECT q1.person_name FROM queue q1
INNER JOIN queue q2 ON q1.turn >= q2.turn
GROUP BY q1.turn
HAVING SUM(q2.weight) <= 1000
ORDER BY q1.turn DESC LIMIT 1;
```

```
-- Optimal — window function O(n log n):
```

```
SELECT person_name FROM (
  SELECT person_name,
         SUM(weight) OVER (
           ORDER BY turn
           ROWS UNBOUNDED PRECEDING
         ) AS cum_weight
  FROM queue
) t
WHERE cum_weight <= 1000
ORDER BY cum_weight DESC
LIMIT 1;
```

Running Sum Pattern

- Self-join: for each person sums all prior weights → $O(n^2)$.
- SUM OVER (ROWS UNBOUNDED PRECEDING): efficient running total $O(n \log n)$.
- Last valid person = highest cumulative weight that is still ≤ 1000 .

#1789

Primary Department for Each Employee

UNION · Window Function

Medium

Pattern	Two cases: explicit primary flag OR only one department
Assessment	CORRECT

● SQL

```
-- Your UNION solution:
SELECT employee_id, department_id FROM employee
WHERE primary_flag = 'Y'
UNION
SELECT employee_id, department_id FROM employee
GROUP BY employee_id
HAVING COUNT(*) = 1;

-- Cleaner – window function avoids UNION:
SELECT employee_id, department_id FROM (
    SELECT employee_id, department_id, primary_flag,
           COUNT(*) OVER (PARTITION BY employee_id) AS dept_count
    FROM employee
) t
WHERE primary_flag = 'Y' OR dept_count = 1;
```

Two-Case Logic

- Case 1: Multiple departments → return the one where primary_flag = Y.
- Case 2: Only ONE department → automatically the primary.
- UNION deduplicates safely here.
- Window function approach avoids UNION and is more concise.

#1045

Customer Who Bought All Products

Relational Division

Medium

Pattern	HAVING COUNT(DISTINCT) = (SELECT COUNT FROM all_items)
Assessment	OPTIMAL

SQL

```
SELECT customer_id
FROM customer
GROUP BY customer_id
HAVING COUNT(DISTINCT product_key) =
       (SELECT COUNT(*) FROM product);
```

Relational Division Pattern

- Classic relational division: "find entities associated with ALL items in a set."
- COUNT(DISTINCT product_key) per customer must equal total products.
- Scalar subquery (SELECT COUNT FROM product) runs once.
- Recurs in interviews: "students enrolled in all courses", "employees with all skills."

#1070

Product Sales Analysis III

Row Constructor Subquery

Medium

Pattern	(col_a, col_b) IN (SELECT col_a, MIN(col_b) ...)
Assessment	OPTIMAL

● SQL

```
SELECT product_id, year AS first_year, quantity, price
FROM   sales
WHERE  (product_id, year) IN (
      SELECT product_id, MIN(year)
      FROM   sales
      GROUP BY product_id
    );
```

Row Constructor Multi-column IN

- (product_id, year) IN (...) matches on TWO columns simultaneously.
- Finds first year of sale per product then returns that full row.
- Alternative: JOIN on (product_id, MIN(year)) as derived table — equally valid.

#1731

Number of Employees Reporting to Each Employee

Self Join · ROUND

Medium

Concept	Count reports and average age per manager
Assessment	OPTIMAL

● SQL

```
SELECT  e1.employee_id,
        e1.name,
        COUNT(e2.reports_to) AS reports_count,
        ROUND(AVG(e2.age))   AS average_age
FROM    employees e1
INNER   JOIN employees e2 ON e1.employee_id = e2.reports_to
GROUP   BY e1.employee_id
ORDER   BY e1.employee_id;
```

Analysis

- INNER JOIN: only managers with at least one direct report appear.
- COUNT(e2.reports_to) counts the number of reports.
- ROUND(AVG(e2.age)) with no decimal places rounds to nearest integer.

#1978

Employees Whose Manager Left

NOT IN · NULL Trap

Medium

Caution

NOT IN with a subquery that returns NULL makes the entire result empty

Assessment

CORRECT — verify manager_id is NOT NULL

● SQL

```
SELECT employee_id
FROM   employees
WHERE  salary < 30000
      AND manager_id NOT IN (
          SELECT employee_id FROM employees
      )
ORDER BY employee_id;
```

NOT IN NULL Trap

- NOT IN (subquery) is safe HERE because employee_id is NOT NULL.
- DANGER: if the subquery returned any NULL, NOT IN would produce empty results.
- Safe alternative: WHERE NOT EXISTS (SELECT 1 FROM employees e2 WHERE e2.employee_id = e.manager_id).
- Always verify the subquery column is NOT NULL when using NOT IN.

CHAPTER VIII ·

Problem Deep Dive — Hard Problems

#185

Department Top Three Salaries

DENSE_RANK · PARTITION BY · CTE

Hard

Pattern	Top-N per group — the most common Hard SQL pattern
Assessment	OPTIMAL

● SQL

```
SELECT Department, Employee, Salary
FROM (
    SELECT d.name AS Department,
           e.name AS Employee,
           e.salary AS Salary,
           DENSE_RANK() OVER (
               PARTITION BY d.id
               ORDER BY e.salary DESC
           ) AS rnk
    FROM employee e
    INNER JOIN department d ON e.departmentid = d.id
) t
WHERE rnk <= 3;
```

Why DENSE_RANK Is Correct Here

- RANK(): gives 1,1,3 for ties — the rank-3 person could be skipped.
- DENSE_RANK(): gives 1,1,2 — always includes 3 distinct salary levels.
- ROW_NUMBER(): unique ranks — would include too many when ties exist at rank 3.
- PARTITION BY d.id resets ranking per department independently.
- This is the gold-standard answer for Top-N per group problems.

#180

Consecutive Numbers

LAG · Self Join · ID Gap Issue

Hard

Bug	id - id = 1 assumes sequential IDs — breaks if IDs have gaps
Fix	Use LAG() which operates on row positions, not ID values

● SQL

```
-- Your solution (assumes consecutive IDs — fragile):
SELECT DISTINCT l1.num AS ConsecutiveNums
FROM   logs l1, logs l2, logs l3
WHERE  l1.id - l2.id = 1 AND l2.id - l3.id = 1
      AND l1.num = l2.num AND l2.num = l3.num;

-- ROBUST — LAG handles ID gaps correctly:
SELECT DISTINCT num AS ConsecutiveNums
FROM (
  SELECT num,
         LAG(num, 1) OVER (ORDER BY id) AS prev1,
         LAG(num, 2) OVER (ORDER BY id) AS prev2
  FROM   logs
) t
WHERE  num = prev1 AND num = prev2;
```

ID Gap Problem

- IDs like 1,2,4,5 — $l1.id - l2.id = 1$ checks ID difference, not row position.
- `LAG(num, 1) OVER (ORDER BY id)` gets the PREVIOUS row's value by row order.
- LAG is immune to ID gaps — always the correct approach for consecutive row problems.
- Also avoids the $O(n^2)$ self-join; LAG is $O(n \log n)$.

#602

Friend Requests II: Who Has the Most Friends

UNION ALL · Bidirectional

Hard

Pattern	UNION ALL both directions then count per person
Assessment	OPTIMAL

SQL

```
SELECT id, COUNT(id) AS num
FROM (
  SELECT requester_id AS id FROM requestaccepted
  UNION ALL
  SELECT acceptor_id AS id FROM requestaccepted
) t
GROUP BY id
ORDER BY num DESC
LIMIT 1;
```

Bidirectional Relationship Pattern

- Friendships are bidirectional — $A \rightarrow B$ and $B \rightarrow A$ both count for both people.
- UNION ALL (not UNION): must include both directions separately.
- UNION would deduplicate — wrong here because we need the full count.
- Edge case: tie for most friends — discuss `RANK()` vs `LIMIT 1` with interviewer.

#585

Investments in 2016

Multi-column IN · Composite Filter

Hard

Pattern	(col_a, col_b) IN (...) matches on two columns simultaneously
Assessment	OPTIMAL

● SQL

```
SELECT ROUND(SUM(tiv_2016), 2) AS tiv_2016
FROM   insurance
WHERE  tiv_2015 IN (
    SELECT tiv_2015 FROM insurance
    GROUP BY tiv_2015 HAVING COUNT(*) > 1
)
AND (lat, lon) IN (
    SELECT lat, lon FROM insurance
    GROUP BY lat, lon HAVING COUNT(*) = 1
);
```

Row Constructor Multi-column IN

- Condition 1: tiv_2015 value is shared by more than one policyholder.
- Condition 2: (lat, lon) location is unique — only one policyholder there.
- (lat, lon) IN (...) matches both columns simultaneously in one expression.
- Both subqueries run once and filter independently — efficient approach.

#626

Exchange Seats

CASE WHEN · MOD · Edge Case

Hard

Pattern	Three-way CASE for swap with edge case on last odd seat
Assessment	OPTIMAL

● SQL

```
-- Your solution (correct):
SELECT
  CASE
    WHEN id = (SELECT MAX(id) FROM seat)
      AND id % 2 = 1 THEN id    -- last seat, odd: keep
    WHEN id % 2 = 1    THEN id + 1 -- odd: swap with next
    ELSE                id - 1  -- even: swap with prev
  END AS id,
  student
FROM seat
ORDER BY id;

-- Alternative with LEAD:
SELECT
  CASE
    WHEN MOD(id, 2) = 0 THEN id - 1
    WHEN LEAD(id) OVER (ORDER BY id) IS NULL THEN id
    ELSE id + 1
  END AS id,
  student
FROM seat ORDER BY id;
```

Three-Case CASE WHEN Logic

- Case 1: Last seat AND odd → keep id (no partner to swap with).
- Case 2: Odd → id+1 (swap forward with next seat).
- Case 3: Even → id-1 (swap backward with previous seat).
- LEAD() approach checks if next row exists — cleaner for the edge case.

#1907

Count Salary Categories

UNION · Ensure All Categories Appear

Hard

Key	SUM(IF(...)) returns 0 (not NULL) — guarantees all 3 categories appear
Assessment	CORRECT

SQL

```
-- Your solution – all 3 rows always present:
SELECT 'Low Salary' AS category,
       SUM(IF(income < 20000, 1, 0)) AS accounts_count
FROM accounts
UNION
SELECT 'Average Salary',
       SUM(IF(income BETWEEN 20000 AND 50000, 1, 0))
FROM accounts
UNION
SELECT 'High Salary',
       SUM(IF(income > 50000, 1, 0))
FROM accounts;
```

Zero-Count Category Problem

- Simple GROUP BY drops categories with zero members — wrong.
- SUM(IF(...)) returns 0 (not NULL) for empty categories — all rows appear.
- General pattern for "include all categories even with 0 count": UNION with SUM(IF) or CROSS JOIN + LEFT JOIN.
- Always ask: "What if a category has 0 members?" — tests edge-case awareness.

#1484

Group Sold Products by the Date

GROUP_CONCAT · MySQL Specific

Hard

BUG	Syntax error in your solution — COUNT and GROUP_CONCAT merged incorrectly
Fix	Separate COUNT(DISTINCT) and GROUP_CONCAT into two distinct columns

SQL

```

-- YOUR SOLUTION HAS A SYNTAX ERROR.
-- Malformed: COUNT(DISTINCT product) as group_concat...

-- CORRECT:
SELECT  sell_date,
        COUNT(DISTINCT product) AS num_sold,
        GROUP_CONCAT(
            DISTINCT product
            ORDER BY product
            SEPARATOR ','
        ) AS products
FROM    activities
GROUP  BY sell_date
ORDER  BY sell_date;

```

GROUP_CONCAT — MySQL-Specific Aggregation

- GROUP_CONCAT(DISTINCT product ORDER BY product SEPARATOR ',') builds a comma-separated string.
- DISTINCT removes duplicates within the concatenation.
- ORDER BY inside GROUP_CONCAT sorts the concatenated values alphabetically.
- PostgreSQL equivalent: STRING_AGG(product, ',' ORDER BY product).

#1517

Find Users Without Valid Emails

REGEXP · Anchor · Character Classes

Hard

Pattern	Regex anchored at start and end with character class constraints
Assessment	OPTIMAL

SQL

```
SELECT *
FROM   users
WHERE  mail REGEXP
      '^[A-Za-z][A-Za-z0-9_.-]*@leetcode[.]com$';
```

REGEXP Pattern Breakdown

- `^[A-Za-z]`: must start with exactly one letter (anchor at start).
- `[A-Za-z0-9_.-]*`: zero or more letters, digits, underscore, period, or hyphen.
- `@leetcode[.]com$`: literal `@leetcode.com` at end (`[.]` = literal dot, not "any char").
- Default MySQL REGEXP is case-insensitive. Add `BINARY` for case-sensitive matching.

#610

Triangle Judgement

CASE WHEN · Triangle Inequality

Hard

Concept	All three triangle inequality conditions must hold simultaneously
Assessment	OPTIMAL

● SQL

```
SELECT x, y, z,
       CASE
         WHEN x + y > z
              AND x + z > y
              AND y + z > x
         THEN 'Yes'
         ELSE 'No'
       END AS triangle
FROM   triangle;
```

Triangle Inequality

- A triangle is valid when each side is strictly less than the sum of the other two.
- All three conditions must hold simultaneously.
- Simple CASE WHEN — no JOIN or subquery needed. Clean single-pass solution.

CHAPTER IX ·

SQL Interview Preparation Guide

SQL Interview Cheat Sheet

Core Query Skeleton

● Full Query Template

```

SELECT  [DISTINCT] col1, AGG(col2) AS alias
FROM    table1 t1
[JOIN]  table2 t2 ON t1.id = t2.fk_id
WHERE   row_condition
GROUP BY col1
HAVING  AGG(col2) > threshold
ORDER BY alias DESC, col1 ASC
LIMIT   n [OFFSET m];

```

Logical Execution Order — Memorise This

- FROM JOIN WHERE GROUP BY HAVING SELECT DISTINCT ORDER BY LIMIT
- WHERE runs before SELECT: you cannot reference a SELECT alias in WHERE.
- HAVING runs after GROUP BY: aggregate functions allowed; not allowed in WHERE.
- ORDER BY runs after SELECT: you CAN reference a SELECT alias in ORDER BY.

Essential Functions Quick Reference

Category	Function / Syntax	Example and Notes
Aggregate	COUNT(*) / COUNT(col) / COUNT(DISTINCT col)	COUNT(*) counts NULLs; COUNT(col) skips NULLs; COUNT(DISTINCT) unique non-NULLs
Aggregate	SUM / AVG / MIN / MAX	All ignore NULLs. ROUND(AVG(col), 2) for 2 decimal places.
Conditional	IF(cond, true_val, false_val)	SUM(IF(state='ok', 1, 0)) — conditional count in one pass
Conditional	CASE WHEN c THEN v ELSE d END	Standard SQL alternative to IF(). Preferred in interviews.
Conditional	IFNULL(val, default) / COALESCE(a,b,c)	IFNULL(bonus, 0). COALESCE returns first non-NULL value.
String	CONCAT / LEFT(s,n) / SUBSTRING(s,pos)	CONCAT(UPPER(LEFT(name,1)), LOWER(SUBSTRING(name,2)))
String	LENGTH(s) / CHAR_LENGTH(s)	LENGTH = bytes. CHAR_LENGTH = characters (safe for Unicode/emoji).
String	LIKE / REGEXP	LIKE '%x%' contains x. REGEXP '(^)DIAB1' anchored pattern.

String	GROUP_CONCAT(DISTINCT col ORDER BY col SEPARATOR ',')	MySQL-only: comma-separated list per group (LC #1484).
Date	DATE_FORMAT(date, '%Y-%m')	Year-month string for grouping. '%Y-%m-%d' full date.
Date	DATEDIFF(a, b)	Integer days between two dates. Positive when a is later than b.
Date	DATE_SUB(date, INTERVAL n DAY)	DATE_SUB('2023-07-27', INTERVAL 29 DAY) gives 2023-06-28.
Date	TIMESTAMPDIFF(MINUTE, a, b)	Minutes between two datetimes. Useful for session detection.
Window	ROW_NUMBER() OVER (PARTITION BY g ORDER BY col)	Always unique rank. No ties ever.
Window	DENSE_RANK() OVER (PARTITION BY g ORDER BY col)	Same rank for ties, no gaps. Best choice for Top-N problems.
Window	RANK() OVER (...)	Same rank for ties but skips next ranks. Rarely the right choice.
Window	LAG(col, n, default) / LEAD(col, n, default)	Value n rows before/after current row in window order.
Window	SUM/AVG OVER (ORDER BY col ROWS BETWEEN n PRECEDING AND CURRENT ROW)	Rolling aggregation. ROWS UNBOUNDED PRECEDING = running total.
Window	MIN/MAX OVER (PARTITION BY col)	Per-group min/max on every row without collapsing groups.
Set Ops	UNION / UNION ALL	UNION deduplicates (adds sort). UNION ALL keeps all rows (faster).
NULL	IS NULL / IS NOT NULL	NEVER use = NULL or != NULL. They always return UNKNOWN.
Math	ROUND(val,n) / FLOOR(x) / CEIL(x) / MOD(val,d)	ROUND(3.145,2)=3.15. FLOOR(3.9)=3. CEIL(3.1)=4. MOD(id,2)=1 odd.

Top 10 SQL Interview Patterns

Pattern	SQL Technique	LeetCode Example
Anti-Join	LEFT JOIN t2 ON ... WHERE t2.id IS NULL	Customers with no orders (LC #1581, #577)
Top-N per Group	DENSE_RANK() OVER(PARTITION BY g ORDER BY v DESC) WHERE rn<=N	Top 3 salaries per dept (LC #185)
Rolling Window	SUM(col) OVER(ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)	7-day moving average (LC #1321)
Relational Division	HAVING COUNT(DISTINCT x) = (SELECT COUNT(*) FROM all_x)	Bought ALL products (LC #1045)
Consecutive / Gap-Island	LAG(val,1) OVER(ORDER BY id) or DATE_SUB(date, INTERVAL rn DAY)	Consecutive numbers, login streaks (LC #180)
Conditional Aggregation (Pivot)	SUM(IF(cond, val, 0)) per group	Monthly transactions (LC #1193, #1907)
Bidirectional Relationship	UNION ALL of both directions then COUNT GROUP BY	Friend count (LC #602)
Date-Range JOIN	JOIN ON date BETWEEN start_date AND end_date	Correct price tier per date (LC #1251, #1164)
All-Combinations (CROSS JOIN)	CROSS JOIN + LEFT JOIN + COUNT (zero-safe)	Student x subject matrix (LC #1280)
NULL-Safe Default	IFNULL(expr, 0) / COALESCE(a,b) on aggregated columns	Products with no sales give 0 avg price (LC #1251, #619)

Interview Questions — Beginner Level

These questions are asked in junior engineer, junior data analyst, and entry-level SQA engineer interviews. You should be able to answer all of them instantly and without hesitation.

Q1: What is the logical order of SQL clause execution?

A: FROM then JOIN then WHERE then GROUP BY then HAVING then SELECT then DISTINCT then ORDER BY then LIMIT. This means WHERE runs before SELECT, so you cannot reference a SELECT alias in WHERE. HAVING runs after GROUP BY so you can use aggregate functions there.

Q2: What is the difference between WHERE and HAVING?

A: WHERE filters individual rows BEFORE GROUP BY and cannot use aggregate functions. HAVING filters groups AFTER GROUP BY and can use aggregate functions. Example: WHERE salary > 5000 filters rows; HAVING AVG(salary) > 5000 filters groups. Writing WHERE COUNT(*) > 5 is a syntax error; HAVING COUNT(*) > 5 is correct.

Q3: What is the difference between INNER JOIN and LEFT JOIN?

A: INNER JOIN returns only rows that match in BOTH tables, dropping unmatched rows from either side. LEFT JOIN returns ALL rows from the left table plus matched rows from the right table. Unmatched right-table columns appear as NULL. Use LEFT JOIN when you want to keep all records from the left table regardless of matches.

Q4: What is the difference between COUNT(*), COUNT(col), and COUNT(DISTINCT col)?

A: COUNT(*) counts all rows including NULLs. COUNT(col) counts only non-NULL values in that column. COUNT(DISTINCT col) counts unique non-NULL values. For column values [1, 2, NULL, 2]: COUNT(*)=4, COUNT(col)=3, COUNT(DISTINCT col)=2.

Q5: How do you find duplicate records in a table?

A: GROUP BY the columns that define uniqueness, then use HAVING COUNT(*) > 1. Example: SELECT email, COUNT(*) AS cnt FROM users GROUP BY email HAVING COUNT(*) > 1. To see the full duplicate rows, join this result back to the original table.

Q6: How do you get the second highest salary?

A: Three approaches: (1) SELECT MAX(salary) FROM employee WHERE salary < (SELECT MAX(salary) FROM employee). (2) SELECT DISTINCT salary FROM employee ORDER BY salary DESC LIMIT 1 OFFSET 1. (3) DENSE_RANK() window function for the Nth highest generalisation. Always wrap in IFNULL to return NULL when no second salary exists.

Q7: What happens when you compare a value with NULL using = or !=?

A: Both return UNKNOWN, not TRUE or FALSE. NULL = 5 gives UNKNOWN. NULL != 5 gives UNKNOWN. WHERE treats UNKNOWN as FALSE so those rows are silently excluded. Always use IS NULL or IS NOT NULL to check for NULL. Use IFNULL(col, default) to substitute a value before comparison.

Q8: What is the difference between UNION and UNION ALL?

A: UNION removes duplicate rows from the combined result and adds a sort and dedup step. UNION ALL keeps all rows including duplicates and is faster. Use UNION ALL when you know there are no duplicates or when duplicates are intentional, for example when counting friend connections from both the requester and acceptor side.

Q9: How do you delete duplicate rows while keeping one?

A: Use a self-join DELETE: `DELETE p1 FROM person p1 INNER JOIN person p2 ON p1.email = p2.email AND p1.id > p2.id`. This keeps the row with the smallest id for each email and deletes all others. Clarify with interviewers: keep smallest id, largest id, or most recent date?

Q10: What is the purpose of GROUP BY?

A: GROUP BY partitions rows into groups based on one or more columns. Aggregate functions then operate independently per group rather than over all rows. All columns in SELECT that are not inside an aggregate function must appear in GROUP BY. MySQL has a loose GROUP BY mode that silently picks arbitrary values for non-aggregated columns — this is a common source of subtle bugs.

Q11: How do you filter results to only show groups with more than N members?

A: Use HAVING after GROUP BY. Example: `SELECT class FROM courses GROUP BY class HAVING COUNT(*) >= 5`. `WHERE COUNT(*) >= 5` is a syntax error because WHERE runs before GROUP BY and aggregate functions are not yet evaluated at that point.

Q12: What is a NULL value and how is it different from zero or an empty string?

A: NULL means the absence of a value — unknown. 0 is a valid integer value. An empty string is a valid string value. NULL in arithmetic always produces NULL: `5 + NULL = NULL`. NULL in comparisons always produces UNKNOWN: `NULL = NULL` is not TRUE. Use `IS NULL` to check for NULL, not `= NULL`.

Interview Questions — Intermediate Level

These questions are asked for mid-level software engineer, data analyst, data engineer, and SQA automation engineer roles. Expect the interviewer to ask you to write actual SQL after your explanation.

Q1: What is the difference between `RANK()`, `DENSE_RANK()`, and `ROW_NUMBER()`?

A: All three are window functions that rank rows within a partition. `ROW_NUMBER` always assigns unique ranks with no ties: 1,2,3,4. `RANK` assigns the same rank to ties and then skips the next positions: 1,1,3,4. `DENSE_RANK` assigns the same rank to ties with no gaps: 1,1,2,3. For Top-N problems always use `DENSE_RANK` because it returns N distinct value levels even when ties exist at the boundary.

Q2: How do you find the top 3 salaries per department?

A: Use `DENSE_RANK` with `PARTITION BY` department: `SELECT dept, emp, salary FROM (SELECT d.name AS dept, e.name AS emp, e.salary, DENSE_RANK() OVER (PARTITION BY d.id ORDER BY e.salary DESC) AS rn FROM employee e JOIN department d ON e.departmentid = d.id) t WHERE rn <= 3`. `DENSE_RANK` ensures three distinct salary levels are returned per department even when multiple employees share a salary.

Q3: What is a correlated subquery and when would you use one?

A: A correlated subquery references a column from the outer query. It executes once per row of the outer query giving $O(n^2)$ complexity. Example: `SELECT name FROM emp e WHERE salary > (SELECT AVG(salary) FROM emp WHERE dept = e.dept)`. Use it for per-row comparisons that cannot be expressed with a plain join. For performance-sensitive queries replace with a window function: `AVG(salary) OVER (PARTITION BY dept)`.

Q4: How do you calculate a 7-day rolling average?

A: First aggregate data by day. Then apply a window function with a frame clause: `AVG(daily_amount) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)`. This gives a 7-row window (6 preceding + current). Skip the first 6 incomplete rows with `LIMIT large_number OFFSET 6`. This is the optimal $O(n \log n)$ approach vs a correlated subquery which is $O(n^2)$.

Q5: What is the difference between a CTE and a subquery?

A: A CTE is defined at the top of the query with the `WITH` clause, is named, and can be referenced multiple times within the same query. A subquery is inline and can become deeply nested and hard to read. Both produce temporary result sets. CTEs do not write to disk by default. Use CTEs for multi-step logic or when the same intermediate result is needed more than once.

Q6: How do you handle NULL values in aggregate functions?

A: All aggregate functions except `COUNT(*)` ignore NULLs automatically. `SUM` of [NULL, 1, 2] = 3 not NULL. `AVG` of [NULL, 4, 6] = 5 using 2 as the denominator. To treat NULLs as a specific value before aggregation use `IFNULL`: `SUM(IFNULL(col, 0))`. To count how many NULLs exist: `COUNT(*)` minus `COUNT(col)`.

Q7: Explain the anti-join pattern with an example.

A: An anti-join finds rows in table A that have NO matching row in table B. Three equivalent approaches: (1) LEFT JOIN + WHERE right.id IS NULL. (2) WHERE NOT EXISTS (SELECT 1 FROM B WHERE B.id = A.id). (3) WHERE A.id NOT IN (SELECT id FROM B) which is dangerous if subquery can return NULLs. The LEFT JOIN approach is generally the most efficient and safest.

Q8: How do you find customers who purchased every product in the catalogue?

A: Use relational division: GROUP BY customer_id then HAVING COUNT(DISTINCT product_key) = (SELECT COUNT(*) FROM product). This works because a customer who bought all N products will have exactly N distinct product keys. This pattern recurs in interviews as students in all courses or employees with all certifications.

Q9: What is the difference between LAG and LEAD window functions?

A: Both access adjacent rows in the window. LAG(col, n, default) returns the value n rows BEFORE the current row in the window ordering. LEAD(col, n, default) returns the value n rows AFTER the current row. The third argument is a default value returned when no adjacent row exists. Common uses: consecutive day comparison, month-over-month growth, streak detection.

Q10: How would you pivot rows into columns in SQL?

A: MySQL has no PIVOT keyword. Use conditional aggregation: SELECT product, SUM(IF(quarter='Q1', revenue, 0)) AS Q1, SUM(IF(quarter='Q2', revenue, 0)) AS Q2 FROM sales GROUP BY product. If the number of distinct column values is unknown at query time you need dynamic SQL using PREPARE and EXECUTE statements.

Q11: What is a self-join and when would you use one?

A: A self-join joins a table to itself using two different aliases. Common use cases: employee-manager hierarchies, consecutive row comparisons, and finding pairs satisfying a condition within the same table. Example finding managers: SELECT m.name AS manager FROM employee e JOIN employee m ON e.manager_id = m.id. Prefer window functions like LAG over self-joins for consecutive row problems because self-joins are $O(n^2)$ while window functions are $O(n \log n)$.

Q12: How would you find employees earning more than the average salary in their department?

A: Window function approach (preferred): SELECT name, salary, dept FROM (SELECT name, salary, dept, AVG(salary) OVER (PARTITION BY dept) AS dept_avg FROM emp) t WHERE salary > dept_avg. Correlated subquery approach: WHERE salary > (SELECT AVG(salary) FROM emp e2 WHERE e2.dept = e.dept). The window function avoids the $O(n^2)$ correlated subquery execution.

Interview Questions — Advanced / FAANG Level

These questions are asked for senior engineer, staff engineer, data engineering lead, and FAANG-level interview rounds. You are expected to write the SQL, discuss optimisation, and explain trade-offs.

Q1: How do you solve the consecutive numbers problem without assuming sequential IDs?

A: Use LAG which operates on row positions not ID values: `SELECT DISTINCT num FROM (SELECT num, LAG(num,1) OVER (ORDER BY id) AS p1, LAG(num,2) OVER (ORDER BY id) AS p2 FROM logs) t WHERE num=p1 AND num=p2`. The id-based self-join approach where `l1.id minus l2.id = 1` breaks silently when IDs have gaps such as 1,2,4,5.

Q2: Explain the gap-and-island problem and how to solve it.

A: Islands are consecutive groups of rows sharing a property; gaps are breaks between them. Standard solution: assign `ROW_NUMBER` per user ordered by date. Compute `DATE_SUB(date, INTERVAL rn DAY)` as a group key. This value is constant within a consecutive island because date and rn both increase by 1. A gap shifts the group key, creating a new distinct value. `GROUP BY (user_id, grp_key)` then `MIN` and `MAX` date gives each island range.

Q3: How would you implement session detection in SQL?

A: A new session starts when the gap to the previous event exceeds a threshold. Step 1: `LAG(event_time)` per user to get previous event time. Step 2: Flag rows where `TIMESTAMPDIFF(MINUTE, prev_time, event_time) > 30` or where `prev_time` IS NULL as `is_new_session = 1` else 0. Step 3: `SUM(is_new_session) OVER (PARTITION BY user_id ORDER BY event_time ROWS UNBOUNDED PRECEDING)` gives a cumulative session ID that increments at each new session.

Q4: What are the performance implications of correlated subquery versus JOIN?

A: A correlated subquery executes once per row of the outer query giving $O(n \times m)$ complexity. A JOIN is executed as a hash join or merge join giving $O(n + m)$ or $O(n \log m)$. For a table with 1 million rows a correlated subquery may run 1 million subqueries while a JOIN processes both tables in a single pass. Always replace correlated subqueries with LEFT JOIN or window functions for large tables.

Q5: How do you calculate month-over-month growth rate in SQL?

A: Step 1: `GROUP BY` month to get the monthly metric. Step 2: `LAG(revenue) OVER (ORDER BY month)` to access the prior month value. Step 3: (current minus previous) times 100.0 divided by previous as `growth_pct`. Handle the first month NULL with `IFNULL(..., 0)`. For year-over-year growth use `LAG(revenue, 12) OVER (ORDER BY month)`.

Q6: How do you find the median salary in MySQL without a MEDIAN function?

A: MySQL has no MEDIAN function. Use `ROW_NUMBER` from both ends: `SELECT AVG(salary) FROM (SELECT salary, ROW_NUMBER() OVER (ORDER BY salary) AS rn, COUNT(*) OVER () AS n FROM employee) t WHERE rn IN (FLOOR((n+1)/2), CEIL((n+1)/2))`. For odd N both FLOOR and CEIL point to the same middle row giving one value. For even N they point to the two middle rows and AVG gives the correct median.

Q7: Explain partition pruning and how it affects query performance.

A: Partition pruning is when the query optimiser eliminates entire table partitions that cannot contain rows matching the WHERE condition. A table partitioned by year with WHERE year = 2023 scans only the 2023 partition. Functions on the partition key such as WHERE YEAR(date) = 2023 disable pruning because the optimiser cannot determine which partition applies. Use WHERE date BETWEEN literal dates instead to enable pruning.

Q8: How would you find users active every single day for the last 30 days?

A: Filter activity to the last 30 days. GROUP BY user_id and count DISTINCT activity dates. Keep users where that count equals exactly 30: `SELECT user_id FROM activity WHERE activity_date >= DATE_SUB(CURDATE(), INTERVAL 29 DAY) GROUP BY user_id HAVING COUNT(DISTINCT activity_date) = 30`. The INTERVAL is 29 days so that including both endpoints gives exactly 30 days.

Q9: What is the difference between ROWS and RANGE in window frame specifications?

A: ROWS counts physical rows regardless of their values. ROWS BETWEEN 1 PRECEDING AND CURRENT ROW always includes exactly 2 rows. RANGE groups rows with identical ORDER BY values into the same boundary. RANGE BETWEEN 1 PRECEDING AND CURRENT ROW includes all rows within 1 unit of the current value. Use ROWS in almost all practical cases. RANGE can produce unexpected results when ORDER BY values have ties.

Q10: How would you detect circular references in a self-referencing hierarchy?

A: Use a recursive CTE with a path tracker. Start from root nodes where parent_id IS NULL. On each recursive step append the child id to the path string. Add WHERE FIND_IN_SET(child.id, path) = 0 to stop when the id already appears in the path. Any node excluded by this condition is part of a circular reference. The MAX recursion depth (default 1000 in MySQL) also acts as a safety net.

Q11: Describe a scenario where denormalisation improves SQL query performance.

A: Normalised schemas minimise redundancy but require many JOINS for analytics queries. At scale with billions of rows these JOINS are expensive even with indexes. Denormalisation pre-joins data into wide tables called fact tables in a star schema. Example: instead of joining product, category, and supplier on every query, maintain a denormalised sales_fact table with product_name and category_name already joined. Trade-off: write operations become more complex and storage increases. Read performance improves dramatically. Used in OLAP systems and data warehouses.

SQL Debugging Checklist

Run through this list whenever a SQL query returns wrong results or throws an error. The majority of production SQL bugs fall into one of these ten categories.

#	Mistake	Symptom	Fix
1	NULL in WHERE filter	col != value silently drops NULL rows	Add OR col IS NULL, or use IFNULL(col, default)
2	LEFT JOIN WHERE on right column	Unmatched NULLs dropped — accidental INNER JOIN	Add OR right.col IS NULL, or move condition to ON clause
3	Aggregate function in WHERE	WHERE COUNT(*) > 5 gives SQL syntax error	Move aggregate condition to HAVING clause
4	RANK instead of DENSE_RANK for Top-N	Tied rows at the boundary are skipped	Use DENSE_RANK for Top-N problems
5	UNION instead of UNION ALL for bidirectional	Duplicate connections deduplicated away	Use UNION ALL when duplicates are intentional
6	id+1 for consecutive rows	Breaks silently when IDs have gaps	Use LAG(col, 1) OVER (ORDER BY id)
7	Empty category dropped by GROUP BY	Zero-count groups missing from result	CROSS JOIN + LEFT JOIN, or UNION with SUM(IF(...))
8	Division by zero in percentage	NULL or error when denominator is zero	NULLIF(denominator, 0) or IF(SUM(units)=0, 0, ...)
9	NOT IN with NULL in subquery	Entire NOT IN result is empty	Use NOT EXISTS, or ensure subquery column is NOT NULL
10	MIN per column pair confusion	MIN(col_a) and MIN(col_b) may come from different rows	Use row constructor: WHERE (id, date) IN (SELECT id, MIN(date) ...)

Structured Revision Plans

1-Day SQL Revision Plan (8 Hours)

Time	Topic	Action
Hour 1-2	Fundamentals + Joins	Read Ch I-III. Focus: NULL traps, LEFT JOIN danger, anti-join, CROSS JOIN.
Hour 3-4	25 Easy Problems	Solve or review all Easy problems. Write the pattern name next to each one.
Hour 5-6	15 Medium Problems	Solve or review all Medium problems. Note every window function optimisation.
Hour 7	10 Hard Problems	Solve or review all Hard problems. Focus on DENSE_RANK, LAG, GROUP_CONCAT.
Hour 8	Interview Prep	Read Beginner and Intermediate Q&A; sections. Memorise Top 10 Patterns table.

3-Day SQL Revision Plan

Day	Morning	Afternoon	Evening
Day 1	Theory Ch I-V: fundamentals, aggregation, joins, subqueries, window functions.	Easy problems 1-15. Focus: NULL, LEFT JOIN, anti-join, string functions.	Easy problems 16-25. Advanced patterns Ch VI: gap-and-island, session, median.
Day 2	Medium 1-8: #176, #1280, #570, #1934, #1193, #1174, #550, #1164.	Medium 9-15. Review all correlated -subquery-to-window-function optimisations.	All 10 Hard problems. Review the 3 bugs: #1174, #180, #1484.
Day 3	Beginner Q&A; (12 questions) - answer aloud without looking at answers.	Intermediate Q&A; (12 questions). Write SQL for each one.	Advanced Q&A; (11 questions). FAANG strategy section. Final cheat sheet review.

7-Day SQL Mastery Plan

Day	Focus	Problems and Goals
1	SQL Fundamentals	SELECT, WHERE, NULL traps, ORDER BY, DISTINCT, execution order. LC: #1757, #595, #584, #1683, #620.
2	Aggregation and Grouping	GROUP BY, HAVING, IF, CASE, conditional aggregation, pivot pattern. LC: #596, #2356, #1633, #1211, #1193.
3	All Join Types	INNER, LEFT, CROSS, SELF, anti-join, date-range join. LC: #1378, #577, #1068, #1075, #1581, #1280, #197, #1661.
4	Subqueries and CTEs	Scalar, row constructor, relational division, WITH clause. LC: #176, #1045, #1070, #1164, #1789, #1978.
5	Window Functions	ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, rolling sum, running total. LC: #185, #180, #550, #1204, #1321.
6	Hard Problems and Advanced Patterns	All 10 Hard problems. Gap-and-island, session detection, median, MoM growth, pivot, full outer join.

7	Mock Interview Day	5 random problems timed at 20 minutes each. Answer all Q&A; sections aloud. Review 10 patterns table.
---	--------------------	---

FAANG-Level SQL Interview Strategy

For top-tier companies SQL interviews evaluate communication, optimisation awareness, edge case thinking, and your ability to reason about scale. A candidate who writes a correct $O(n^2)$ query and says nothing will score lower than one who writes it, identifies the weakness, and proposes the $O(n \log n)$ window function alternative.

1. Explain your approach before writing

Verbalise your plan first. Say: I will use `DENSE_RANK` partitioned by department ordered by salary descending, then filter where rank is 3 or less. This signals structured thinking and gives the interviewer a chance to redirect you before you spend time writing a query in the wrong direction.

2. Write clean and readable SQL

Use uppercase keywords. Align `ON`, `AND`, `WHERE` clauses vertically. Use meaningful table aliases: `e` and `d` not `t1` and `t2`. Add brief inline comments for non-obvious logic. Readable SQL demonstrates engineering maturity and is easier to debug during the interview.

3. State time complexity after writing

After getting a correct query say: this self-join is $O(n^2)$ for large tables. I would replace it with `LAG` which is $O(n \log n)$ and also handles ID gaps correctly. Demonstrating performance awareness without being prompted is what separates mid-level from senior candidates.

4. Present at least two approaches per problem

State the trade-off explicitly: the subquery approach is easier to read but $O(n^2)$. The window function approach is $O(n \log n)$ and scales to millions of rows. For this table size I would use the window function. Interviewers reward comparative thinking over single-track solutions.

5. Handle edge cases proactively

Before declaring your solution complete ask yourself: what if the table is empty? What if all values are `NULL`? What if there are ties at the ranking boundary? What if the denominator is zero? Mention these aloud. Addressing edge cases without prompting is a strong positive signal.

6. Discuss index strategy after the query

After writing a correct and optimised query add: to make this efficient at scale I would add a composite index on `department_id` and `salary` to support the `PARTITION BY` and `ORDER BY` in the window function. I would run `EXPLAIN` to verify the index is used.

7. Know schema design basics

FAANG often asks about 1NF 2NF 3NF normalisation. Star schema versus snowflake schema for data warehouses. When to denormalise for read performance. Trade-offs between OLTP which is normalised and write-optimised versus OLAP which is denormalised and read-optimised.

8. Practice explaining aloud daily

Record yourself solving a problem and watch it back. Do you explain what you are typing or just type silently? Interviewers evaluate communication as heavily as correctness. A candidate who types silently for five minutes is a red flag regardless of the solution quality.

SQL Optimisation Guide

Index Usage Principles

```

● EXPLAIN and Index Strategy

-- Check query execution plan:
EXPLAIN SELECT * FROM users WHERE email = 'test@example.com';

-- Key columns to read in EXPLAIN output:
-- type: ALL = full table scan (bad). ref or range = index used (good).
-- key:   which index is being used. NULL means no index.
-- rows:  estimated rows scanned. Lower is better.

-- Create indexes for common query patterns:
CREATE INDEX idx_email      ON users(email);
CREATE INDEX idx_dept_sal ON employees(department_id, salary);

-- Avoid functions on indexed columns in WHERE:
WHERE YEAR(created_at) = 2023      -- BAD: full scan, index unused
WHERE created_at BETWEEN          -- GOOD: index range scan
    '2023-01-01' AND '2023-12-31'

-- Composite index column order matters:
-- Index on (dept, salary) helps: WHERE dept=X ORDER BY salary
-- Index on (salary, dept) does NOT help that same query pattern.
```

Optimisation Rule	Why It Matters	Example
Filter early with WHERE before JOIN	Fewer rows to join, smaller intermediate result sets	WHERE status='active' before a JOIN cuts rows significantly
JOIN instead of correlated subquery	Correlated subquery runs N times giving O(n squared)	Replace per-row AVG subquery with AVG() OVER (PARTITION BY dept)
Window functions over self-joins	Self-join running total is O(n squared); window is O(n log n)	SUM OVER (ROWS UNBOUNDED PRECEDING) replaces JOIN ON id >= id
SELECT only needed columns	Reduces I/O and enables covering index reads	SELECT id, name instead of SELECT *
Avoid functions on indexed columns	Disables index usage, forces full table scan	WHERE YEAR(date)=2023 is bad. WHERE date BETWEEN dates is good.
EXISTS vs IN for large subquery	EXISTS short-circuits on first match, IN materialises full subquery	WHERE EXISTS (SELECT 1 FROM t WHERE t.id = e.id)
UNION ALL over UNION when duplicates acceptable	UNION adds a sort and dedup step costing extra time	Bidirectional friendship count always uses UNION ALL
Partition large tables by date or region	Partition pruning skips irrelevant partitions entirely	Partitioning by year means a 2023 query scans only the 2023 partition

CHAPTER X ·

Key Takeaways and Repository Audit Summary

- 1 NULL Handling Is the Number One Interview Trap**
NULL != value evaluates to UNKNOWN, not FALSE, so those rows are silently dropped by WHERE. Always use IS NULL or IS NOT NULL. Use IFNULL or COALESCE to provide defaults. Problems affected: LC #584, #577, #1978, #619, #176.
- 2 LEFT JOIN Plus WHERE on Right Column Is an Accidental INNER JOIN**
Filtering on a right-table column in WHERE drops the unmatched NULL rows. Fix by adding OR right.col IS NULL to keep them, or move the condition to the ON clause where it does not drop unmatched rows.
- 3 DENSE_RANK Is the Correct Function for Top-N per Group**
PARTITION BY group, ORDER BY value DESC, WHERE $rnk \leq N$. DENSE_RANK gives 1,1,2 for tied values and always returns N distinct value levels. RANK gives 1,1,3 and may skip the boundary rank. ROW_NUMBER gives unique ranks and includes too many tied rows at the boundary.
- 4 Window Functions Beat Self-Joins on Performance**
Replace consecutive-row self-joins with LAG or LEAD. These also handle ID gaps correctly, unlike id minus id approaches. Replace running-total self-joins with SUM OVER ROWS UNBOUNDED PRECEDING. $O(n \log n)$ versus $O(n^2)$ — always choose window functions at scale.
- 5 CROSS JOIN Generates All-Combinations for Zero-Safe Counts**
CROSS JOIN produces every combination as a Cartesian product. Then LEFT JOIN for optional matching data and COUNT for zero-safe results. Essential for student times subject matrices and similar all-pairs problems. Without CROSS JOIN, groups with zero counts disappear from the result.
- 6 UNION ALL for Bidirectional Relationships**
Friendships, messages, and social graph connections are bidirectional. UNION ALL includes both directions separately for correct counts. UNION deduplicates and would give wrong friendship counts. LC #602 is the definitive example of this pattern.
- 7 DATE_FORMAT DATEDIFF and DATE_SUB for All Date Logic**
DATE_FORMAT(date, percent-Y-percent-m) gives year-month strings for grouping. DATEDIFF(a, b) gives integer days between dates. DATE_SUB handles relative date ranges. Use LAG with DATEDIFF for consecutive calendar day comparisons, not id+1.
- 8 IFNULL and COALESCE Prevent NULL Propagation**
NULL in any arithmetic produces NULL. IFNULL(SUM(amount), 0) prevents NULL averages when no rows exist. Always wrap division when the denominator might be zero or NULL. COALESCE(a, b, c) returns the first non-NULL value from its arguments.

9 Relational Division Finds Entities Associated with All Items

Pattern: `HAVING COUNT(DISTINCT x) = (SELECT COUNT(*) FROM all_x)`. This solves customers who bought every product, students enrolled in every course, and employees with every required certification. It is a recurring data analyst and backend interview question.

10 Gap-and-Island Identifies Consecutive Groups

Assign `ROW_NUMBER` per group ordered by the sequence. Subtract that row number from the sequence value to get a constant group key within each island. `GROUP BY` the group key to find each island range. LC #180 is the simplest variant. Login streak problems are the most common real-world use.

11 Session Detection Uses Cumulative Sum of New-Session Flags

Flag each row where the gap to the previous event exceeds the threshold. `SUM` of those flags `OVER ROWS UNBOUNDED PRECEDING` gives a monotonically increasing session ID. This is the standard technique used by web analytics platforms.

12 Always Mention Edge Cases Proactively

Empty tables, all-NULL columns, ties in ranking, single-row tables, division by zero. Stating these aloud without being asked is what distinguishes senior candidates from junior ones in every SQL interview at every company.

Final Assessment — Repository Audit

- Total: 50 out of 50 problems solved.
- Correct: 47 out of 50 (94 percent). Optimal: 38 out of 50 (76 percent).
- Bugs found: LC #1174 MIN pair mismatch, #180 ID gap assumption, #1484 syntax error.
- Strengths: strong fundamentals, consistent NULL awareness, correct window function usage.
- Areas to improve: use `LAG` over self-join for consecutive rows; replace correlated subqueries with window functions for rolling sums.
- Overall assessment: ready for mid-level interviews. Five to seven extra days on window functions and advanced patterns recommended before FAANG-level interviews.

Metric	Result
Total Problems Analysed	50 out of 50
Correct Solutions	47 out of 50 (94 percent)
Optimal Solutions	38 out of 50 (76 percent)
Bugs Identified and Fixed	3 — LC #1174, #180, #1484
NULL Handling Problems	8 problems tested this concept
Window Function Problems	7 problems used window functions
Most Used Pattern	Joins across 20 problems

Hardest Patterns	Relational Division, Consecutive Rows, Rolling Windows
Advanced Patterns Covered	Gap-and-Island, Session Detection, Median, MoM Growth, Pivot, Full Outer Join
Interview Q and A	12 Beginner + 12 Intermediate + 11 Advanced = 35 total questions
Overall Level	Strong Mid-Level SQL Engineer — Ready for Production Interviews